COMSATS University Islamabad (CUI)

Software Design Description
(SDS DOCUMENT)

for

PodVerse

Version 1.0

By

Rabeea Chughtai **CIIT/SP22-BCS-079/ISB**

Sadia Imran **CIIT/SP22-BCS-087/ISB**

Supervisor

Ms. Saadia Maqbool

Bachelor of Science in Computer Science (2022-2026)

Table of Contents

Revision History ..... **iii**

Application Evaluation History ..... **iv**

1. Introduction **1**

1.1 Scope 1

2. Design Methodology and Software Process Model **4**

2.1 Design Methodology: 4

2.2 Software Process Model: 5

3. System Overview **5**

3.1 Architectural Design 5

4. Design Models **8**

4.1 Activity Diagrams 8

4.2 Class Diagram 22

4.3 Sequence Diagrams 2

4.4 State Transition Diagrams 12

5. Data Design **14**

5.1 Data Dictionary 14

6. Implementation **20**

6.1 Algorithm 20

6.2 External APIs/SDKs 22

6.3 User Interface 22

6.4 Deployment 25

7. Testing and Evaluation **26**

7.1 Unit Testing 26

7.2 Functional Testing 26

7.3 Business Rules Testing 27

7.4 Integration Testing 27

Appendix D: General Coding Standards & Guidelines 28

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) ***include the ones given at scope time both in doc and presentation**	**Action Taken**

Supervised by

Supervisor's Name

Signature_____

1. Introduction

PodVerse is an AI-powered podcast creation platform developed to simplify and modernize the podcast production journey for small business owners, entrepreneurs, and content creators. By eliminating technical barriers such as high production costs, complex editing processes, and challenges in finding credible guests, PodVerse enables creators to produce engaging, high-quality podcasts effortlessly. The system integrates advanced technologies like voice cloning, real-time AI interactions, and customizable avatars, providing users with a centralized solution for planning, recording, editing, and publishing their podcast content. With a user-friendly interface and modules tailored for automation and personalization, PodVerse ensures seamless experience, empowering creators to build consistent content and strengthen audience engagement.

1.1 Scope

The scope of PodVerse is covered under the following modules:

1.1.1 Module 1: User Authentication & Profile Management

This module manages authentication, profile customization, and dashboard access for secure login and content engagement.
FE-1: Account creation via email with verification.
FE-2: Secure login via email/password.
FE-3: Password recovery via email verification.
FE-4: Secure Logout.
FE-5: Creator's dashboard for sample podcasts and AI Agent management.
FE-6: View and edit profiles.
FE-7: Activity feeds with updates from followed creators.

1.1.2 Module 2: Voice Cloning

This module allows users to create and manage voice samples for cloning.
FE-1: Enabling the recording or upload of voice samples for cloning.
FE-2: Allowing users to preview voice samples to check on the quality.
FE-3: Allow users to re-record or re-upload if the sample is not as desired.
FE-4: Allow the availability of a library with voices.

1.1.3 Module 3: Knowledge Base Setup

This module facilitates the setup and management of a knowledge base for retrieval.
FE-1: Allow users to upload data in PDFs, URLs, CSV, or JSON.
FE-2: Data is cleaned, processed, and split into meaningful chunks for easy handling.
FE-3: Tag chunks with metadata and link to original sources.
FE-4: Store chunks in a database and generate vector embeddings for better retrieval.

1.1.4 Module 4: RAG Retrieval

This module enhances the retrieval of accurate and relevant information from the knowledge base.
FE-1: Convert queries into vectors to determine purpose.
FE-2: Match vectors with knowledge base data for accurate responses.
FE-3: Re-rank data by relevance for better answers.
FE-4: Provides fact-based correct answers for podcasts.
FE-5: Optimized for large datasets without loss of performance.
FE-6: Allow feedback to enhance agent response quality.

1.1.5 Module 5: Agent Customization

This module allows users to personalize their AI agents.
FE-1: Users can assign a custom name to their AI agent.
FE-2: Users can choose from various voice options to suit their podcast.
FE-3: Users can select or upload an avatar to visually represent their AI agent.

1.1.6 Module 6: Podcast Audio Creation

This module provides tools for creating and publishing podcasts.
FE-1: Let users start dialogues with a customized AI agent.
FE-2: Conversations recorded for user and AI agent.
FE-3: Users allowed minimal editing to trim and splice to rearrange things.
FE-4: Allow users the ability to preview edited podcasts for flow and polish.
FE-5: Export podcasts in MP3/WAV.
FE-6: Save recordings for future use.

1.1.7 Module 7: AI Host

This module provides an AI host to manage the flow of podcasts.
FE-1: AI Host presents the flow of podcasts, introduces the episodes and closes with a summary.
FE-2: Tracks podcast time and gives reminders for the schedule.
FE-3: Facilitates smooth transitions between guests or topics.
FE-4: Provides summary prompts to summarize key points for clarity and engagement.

1.1.8 Module 8: AI Guest

This module provides an AI guest to enhance podcast discussions.
FE-1: AI Guest responds to real-time queries using dynamic responses.
FE-2: AI Guest shares appropriate stories to add depth to the discussion.
FE-3: AI Guest gives back comments/reactions to facilitate engagement.

1.1.9 Module 9: Podcast Video Creation & Publishing

This module enables the creation of video content for podcasts, and AI-generated text summaries.
FE-1: Users select avatars for each of the speakers.
FE-2: The system syncs based on the audio of the podcast.
FE-3: Users can choose from the pre-defined templates and visuals for the video.
FE-4: Pre-designed split-screen video template with two avatars.
FE-5: Exports the final video in MP4 format or publish the final video.
FE-6: Allow creators to rate podcast sessions.
FE-7: Provide written feedback on conversation skills, voice clarity, and AI accuracy.

1.1.10 Module 10: Subscription & Payment

This module manages subscription and payment processes.
FE-1: Let users access some free, limited content.
FE-2: After the limit, users are prompted to subscribe for continued access.
FE-3: Secure payment gateway integration supporting credit/debit cards.
FE-4: User can control subscriptions, see payment history, change methods.
FE-5: Give reminders before expiration with options of auto-renewal.

1.1.11 Module 11: FAQs Chatbot

This module provides a chatbot to answer frequently asked questions.
FE-1: Users can ask about podcasting, AI customization, or troubleshooting.
FE-2: The chatbot retrieves relevant answers from the FAQs and support materials.
FE-3: Provides concise, direct responses for quick problem resolution.

1.1.12 Module 12: Audience Engagement and Communication

This module enables audience interaction, providing a communication platform, and podcast feedback to enhance creator engagement.
FE-1: Notifications for creators about audience interactions.

FE-2: Highlighting top creators and trending content.
FE-3: Integrated communication platform for user-to-user messaging and calls.
FE-4: Generate and display a text-based summary of podcast episodes.

1.1.13 Module 13: Admin Panel & Analytics

This module provides administrative functionalities, including user management, content moderation, analytics, and system configuration.
FE-1: Allow admin to view, search, deactivate, reactivate, or delete user accounts
FE-2: Enable admin to review reported podcasts and comments.
FE-3: Display live statistics on active users, total podcasts created, trending content, user engagement heatmaps, and demographics.
FE-4: Allow admin to track active subscriptions, process refunds, and manage payment disputes.
FE-5: Provide insights into user engagement, podcast performance, and trending topics.

2. Design Methodology and Software Process Model

2.1 Design Methodology:

In developing PodVerse, we've chosen an object-oriented approach.

2.1.1 Rationale behind this approach:

Here are the main reasons for choosing this approach:

Modularity and Reusability:
For PodVerse, which has many features like user authentication and AI-driven podcast creation, having a modular structure is important. Each class can focus on a specific task, making the code easier to manage and less likely to have errors.

Efficiency in Development:
The object-oriented approach allows for rapid development. By breaking down the application into smaller, reusable classes, our team can work on different parts of the app at the same time.

Better Debugging and Testing:
Object-oriented programming allows for testing and debugging of individual classes separately. This means each part of PodVerse can be thoroughly tested before integration, reducing bugs and ensuring higher code quality.

2.2 Software Process Model:

In developing PodVerse, we are following the Incremental model.

2.2.1 Rationale behind this model:

Here are the main reasons for choosing this model:

Step-by-Step Progress:
The Incremental model allows the project to grow through iterative cycles. With each cycle, new features are added, tested, and refined. This ensures that the system is developed and tested in steps, enabling early delivery of core functionalities while allowing for improvements in later cycles.

Feedback Integration:
By delivering the system in steps, we can gather feedback from users at each stage. Early and continuous feedback helps identify any issues or areas for improvement, allowing us to refine and enhance the system iteratively.

Flexibility in Adding Features:
The incremental model provides flexibility in adding new features later. Modules like AI Host, AI Guest, and Podcast Video Creation can be developed and integrated one by one. This approach allows us to adapt to changing requirements.

Continuous Testing and Quality Assurance:
Each step is thoroughly tested before moving on to the next. This ensures that any bugs or issues are identified and resolved early, maintaining the overall quality of the system. Continuous testing helps in building a stable and reliable application, reducing the risk of major defects, and ensuring a smooth user experience.

3. System Overview

PodVerse is an innovative platform designed to simplify podcast creation and production for small business owners, entrepreneurs, and content creators. It addresses common challenges such as high production costs, technical complexities, and securing credible guests. PodVerse enables users to create high-quality podcasts effortlessly, even without extensive technical skills. The platform offers features like voice cloning, editing, and publishing, making the production process seamless. With AI voices and language models, users can create engaging and dynamic podcasts without needing real hosts or guests. PodVerse is built to help creators produce consistent content, grow their brand, and engage effectively with their audience.

3.1 Architectural Design

The architectural design of PodVerse is modular, ensuring that each component of the system works together to achieve the overall functionality. This section provides a high-level overview of how the system’s modules collaborate to deliver the desired features.

3.1.1 Box and Line Diagram

The following diagram shows the modules and their relationships with each other. This diagram is at a high level of abstraction and gives an overview of how different modules of the system interact to achieve the desired functionality.

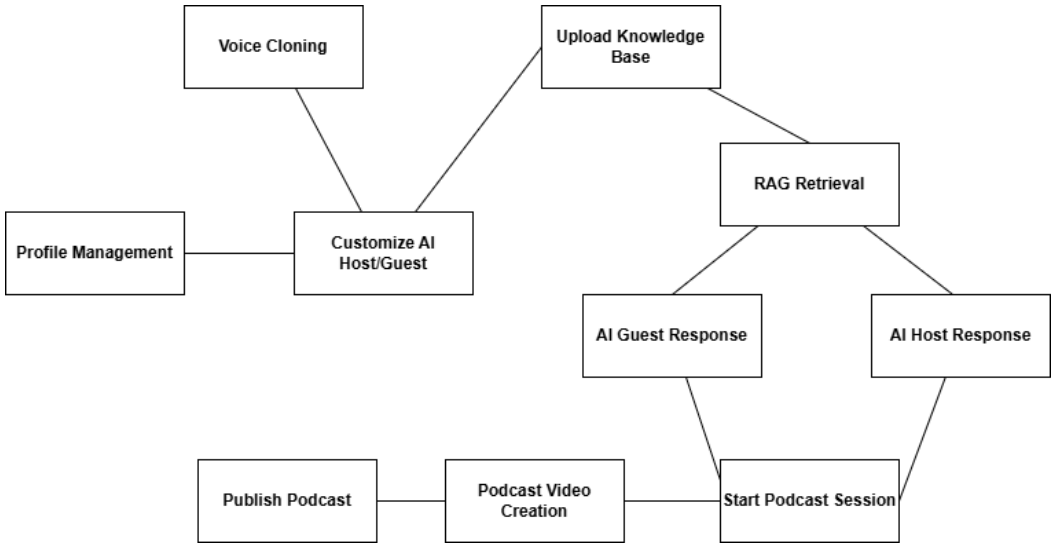


Figure 1: Box and Line Diagram of PodVerse

3.1.2 Layered Architecture

We have adopted a Layered Architecture for the PodVerse application. This architecture style emphasizes the separation of concerns and facilitates modularity.

Presentation Layer (View):

This layer includes the user interface components that interact with the users, such as AudioCreation_View, VideoCreation_View, and ProfileSetting_View. It captures user input and displays output, enabling smooth navigation and interaction within the system.

Application Layer (Controller):

Controllers like AudioCreation_Controller, VoiceCloning_Controller, and Auth_Controller handle user actions, process inputs, coordinate between the view and model, and manage application logic such as podcast creation and authentication.

Business Logic Layer (Model):

This layer contains core logic and data models such as User, Podcast, VoiceSample, and Rating. It manages processing rules, validations, AI interactions, and handles the overall behavior of the system.

AI Services & External Layer:

This layer integrates external services like the Voice Cloning Engine, LLMs, RAG Retrieval, and Subscription APIs. It has intelligent features such as AI guest responses, voice synthesis, and content recommendations.

Data Access Layer:

It abstracts database interactions and performs CRUD operations through repositories. It accesses data stored in relational databases, vector stores, and cloud storage to support both structured and unstructured data needs.

Database Layer:

This layer consists of the actual storage systems, including relational databases for core data, vector stores for AI embeddings, and cloud storage for media files like audio and video podcasts.

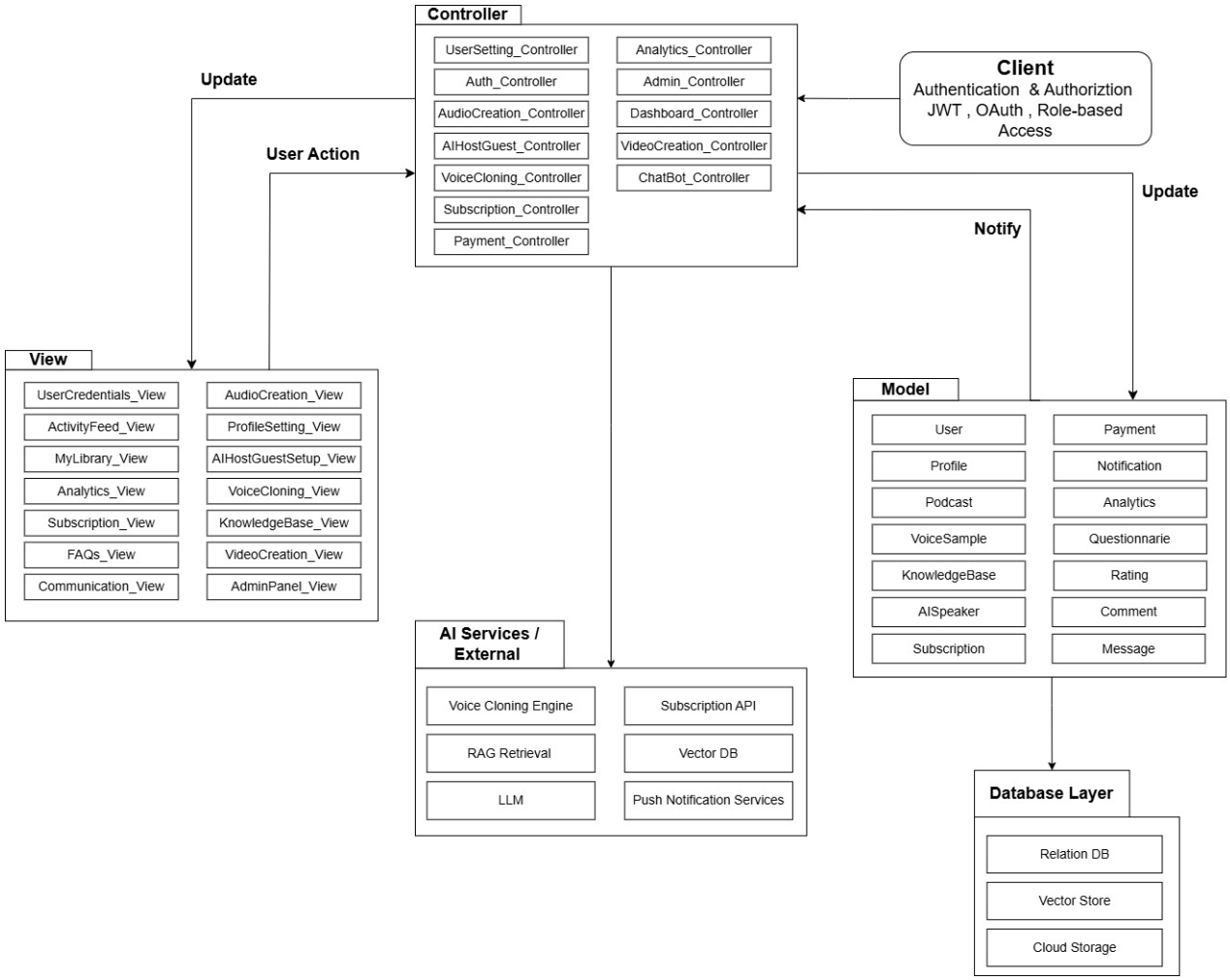


Figure 2: Architecture Diagram for PodVerse

4. Design Models

This section shows the design models for the system.

4.1 Activity Diagrams

4.1.1 Register Account

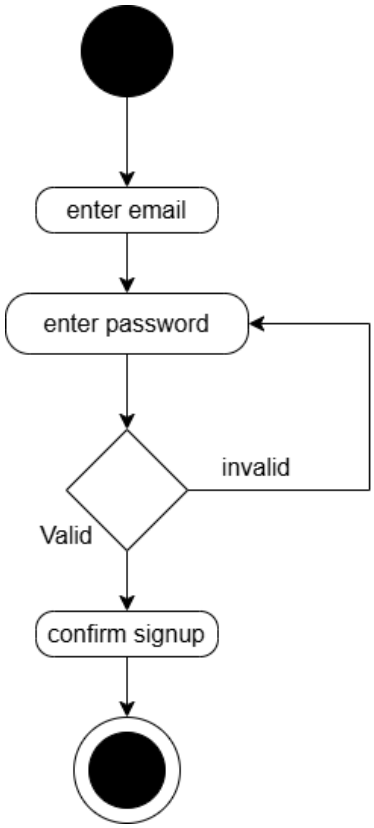


Figure 3: Activity Diagram for Sign up

4.1.2 Login

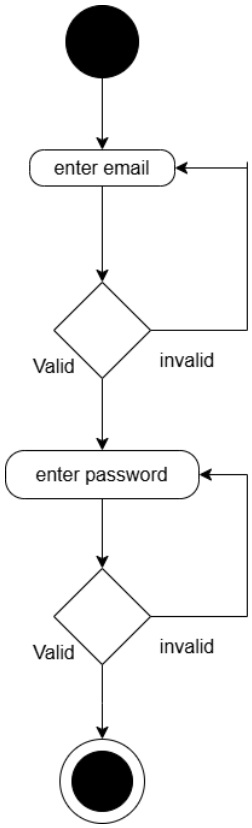


Figure 4: Activity Diagram for Sign in

4.1.3 User Set Profile

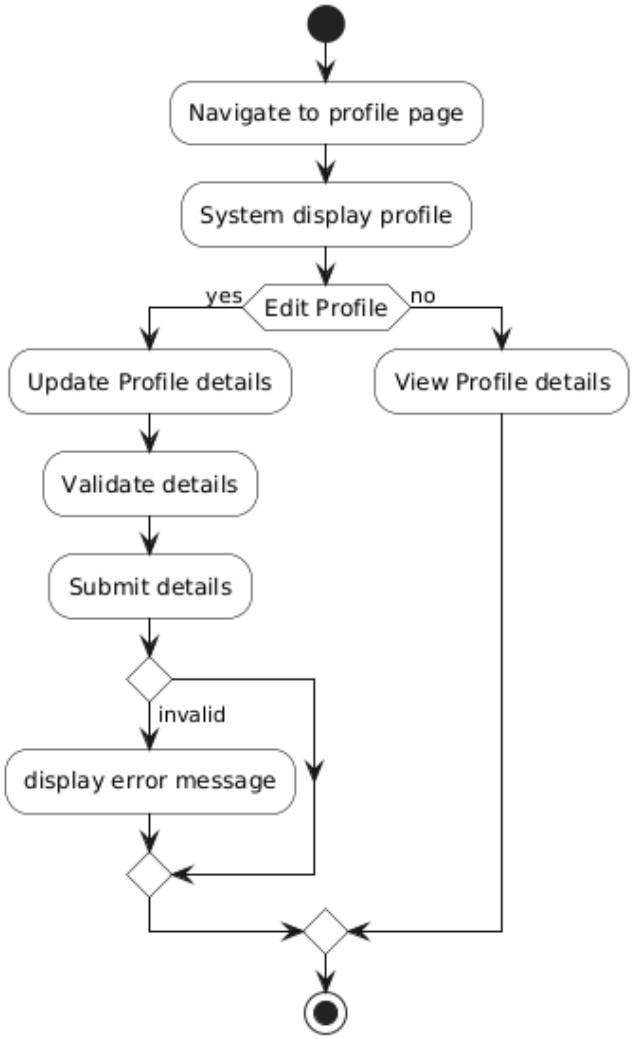


Figure 5: Activity Diagram for User Profile Setup

4.1.4 Voice Cloning

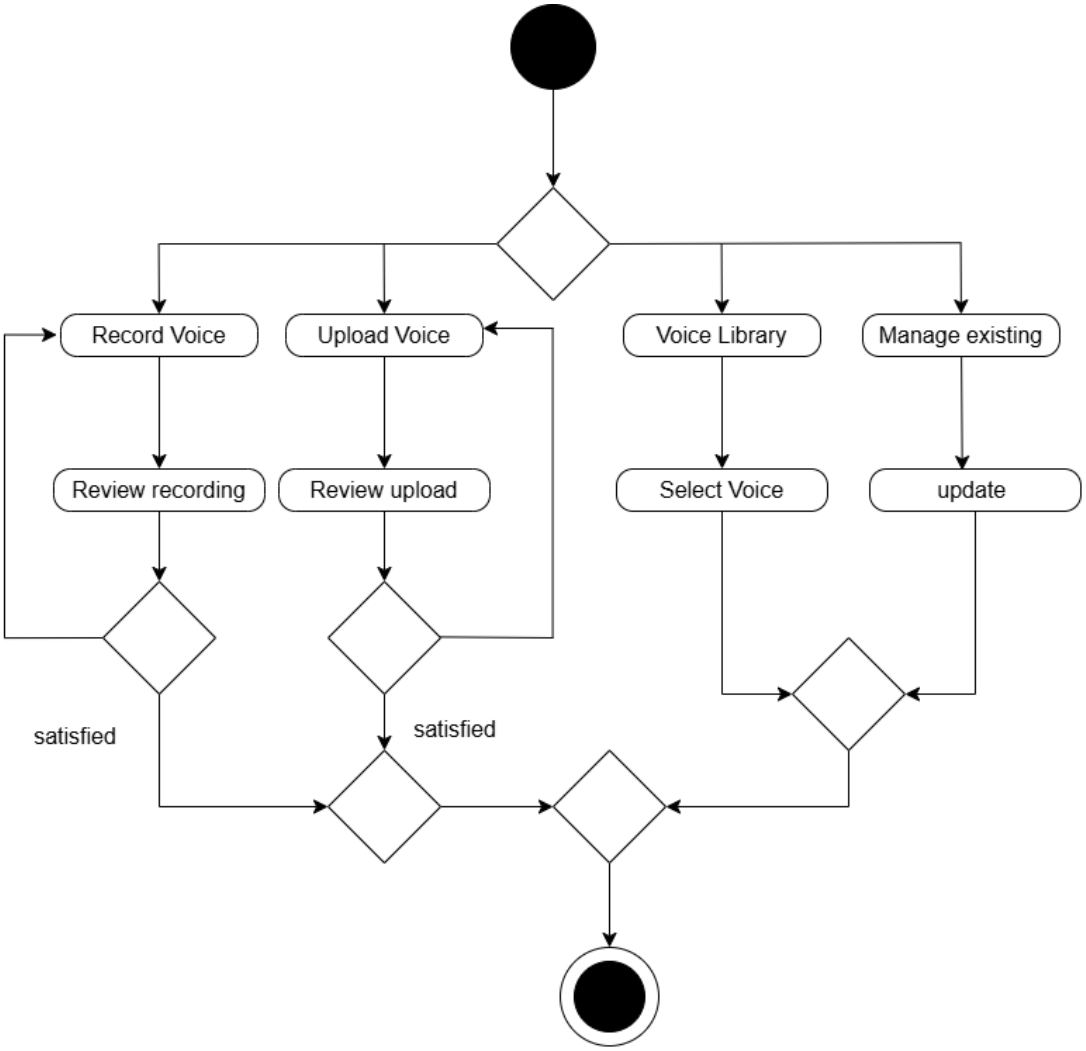


Figure 6: Activity Diagram for Voice Cloning

4.1.5 Knowledge Base Setup

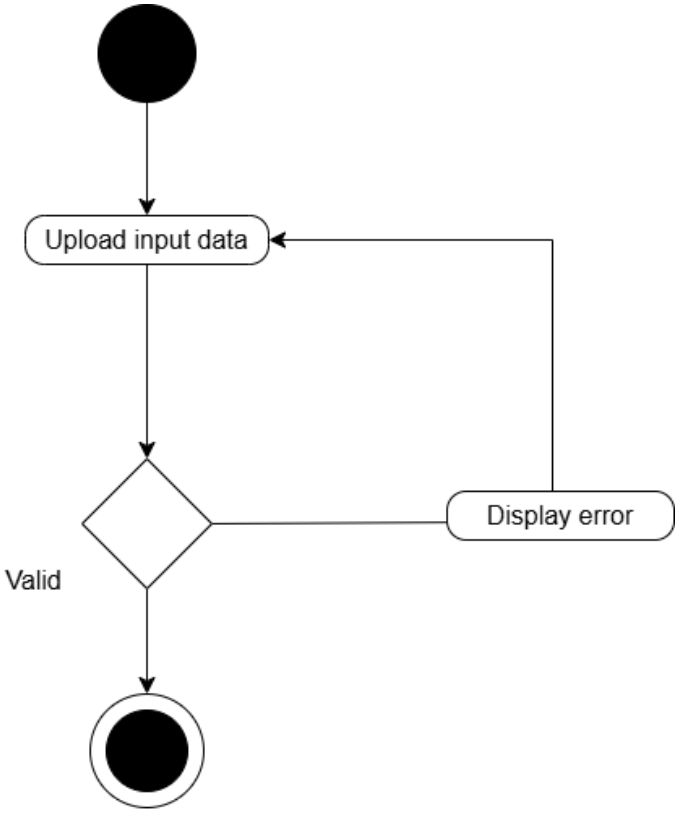


Figure 7: Activity Diagram for Knowledge Base Setup

4.1.6 Agent Customization

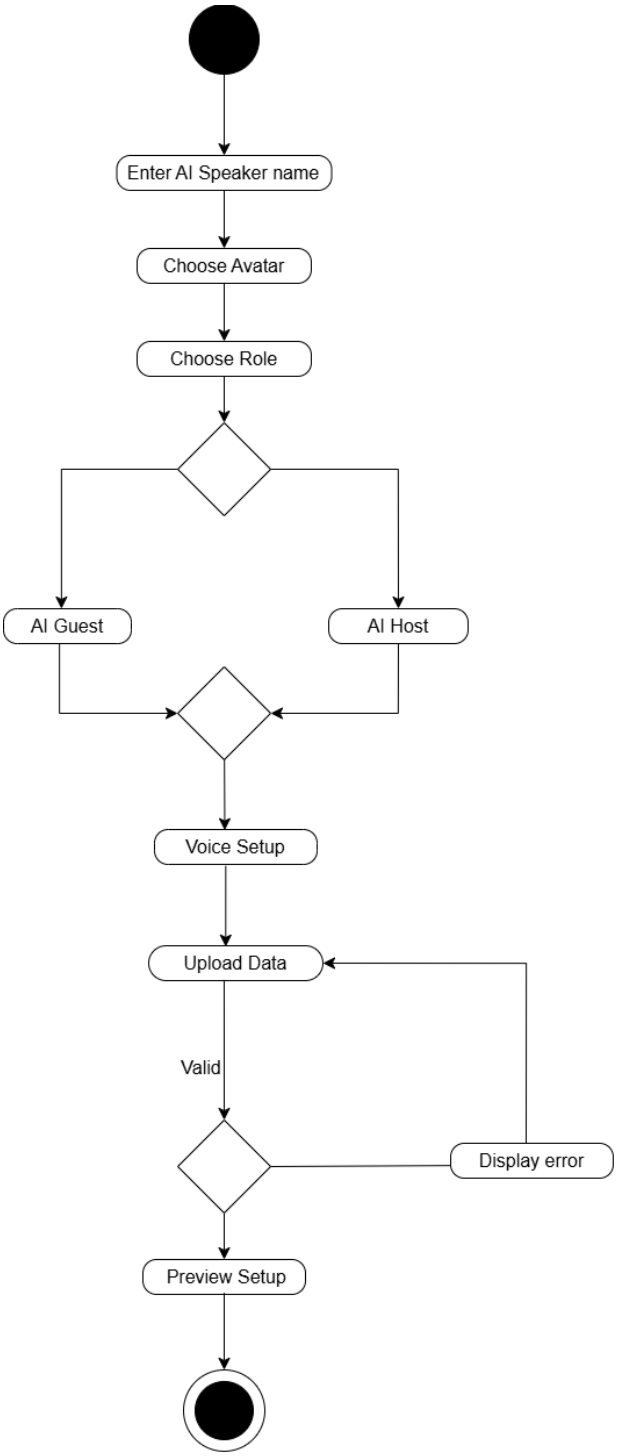


Figure 8: Activity Diagram for Agent Customization

4.1.7 Podcast Audio Creation

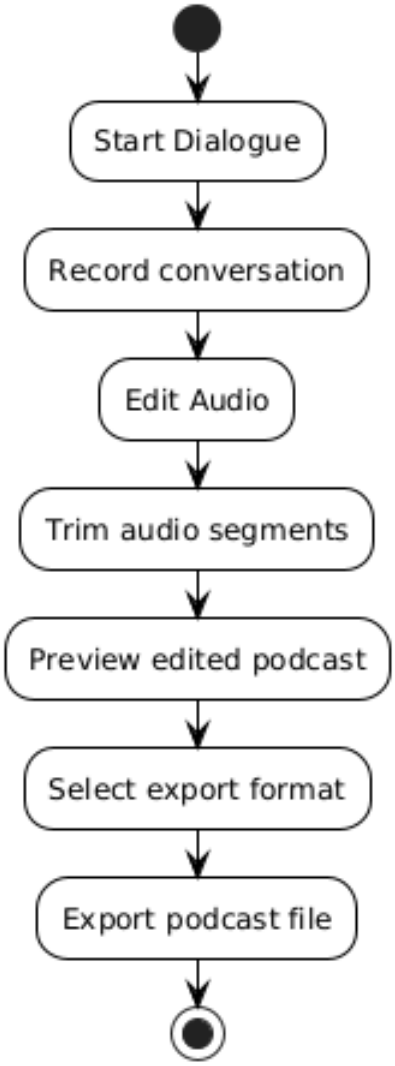


Figure 9: Activity Diagram for Podcast Audio Creation

4.1.8 Podcast Video Creation

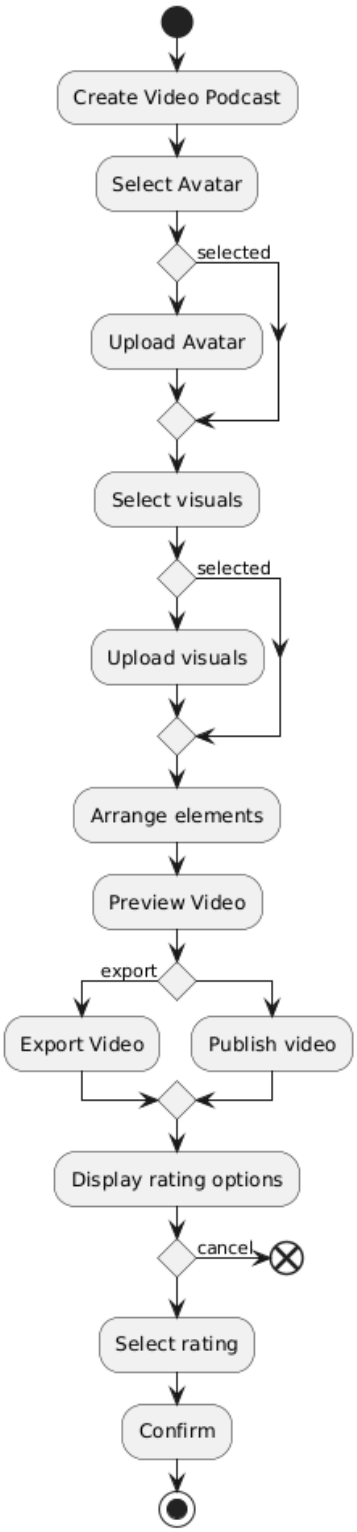


Figure 10: Activity Diagram for Podcast Video Creation

4.1.9 Request Podcast Summary

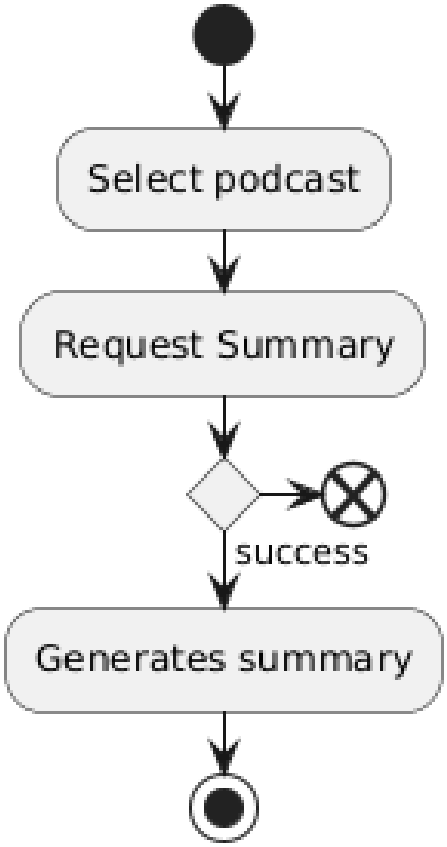


Figure 11: Activity Diagram for Podcast Summary

4.1.10 Subscription and Payment

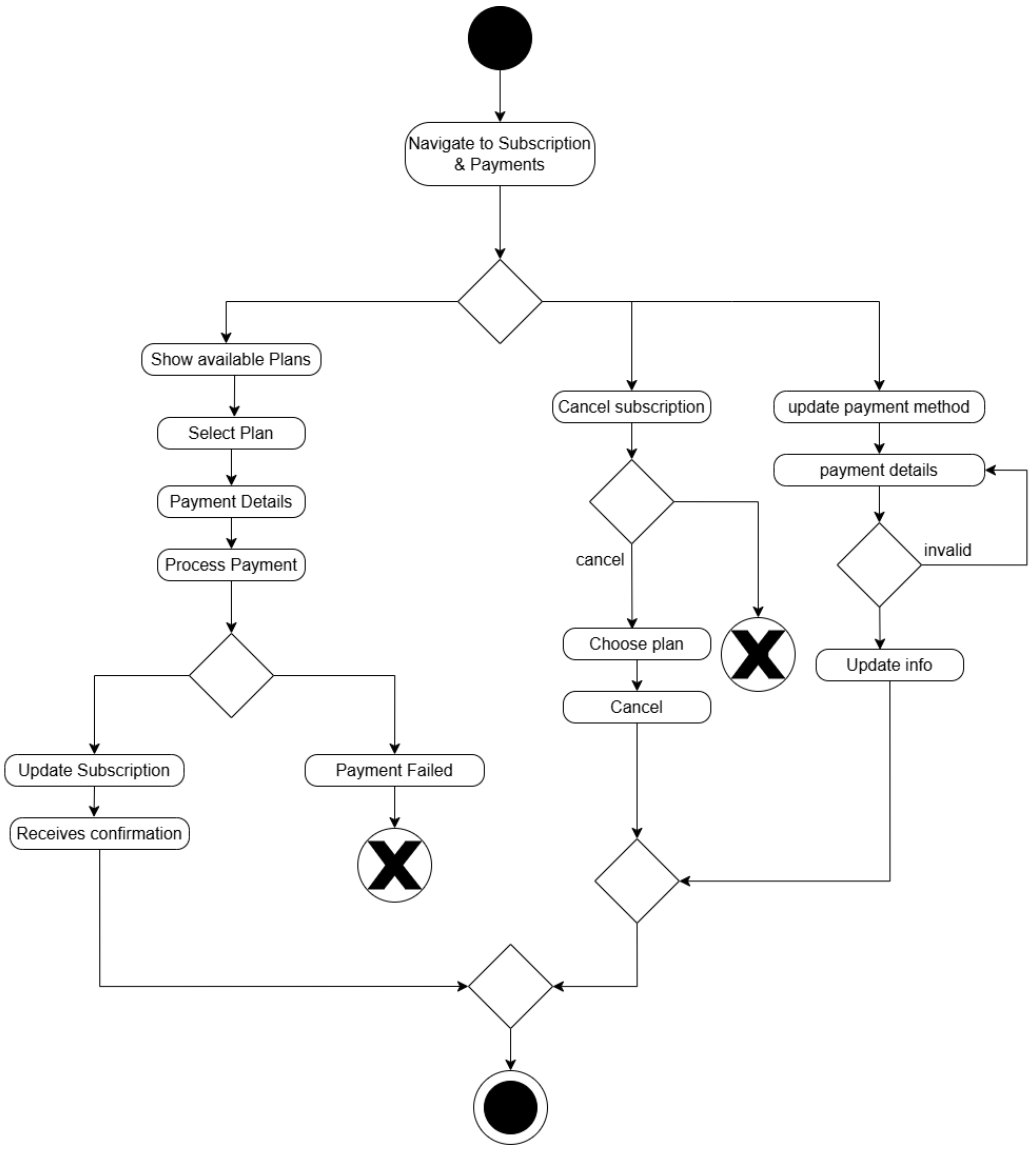


Figure 12: Activity Diagram for Subscription and Payment

4.1.11 FAQs Chatbot

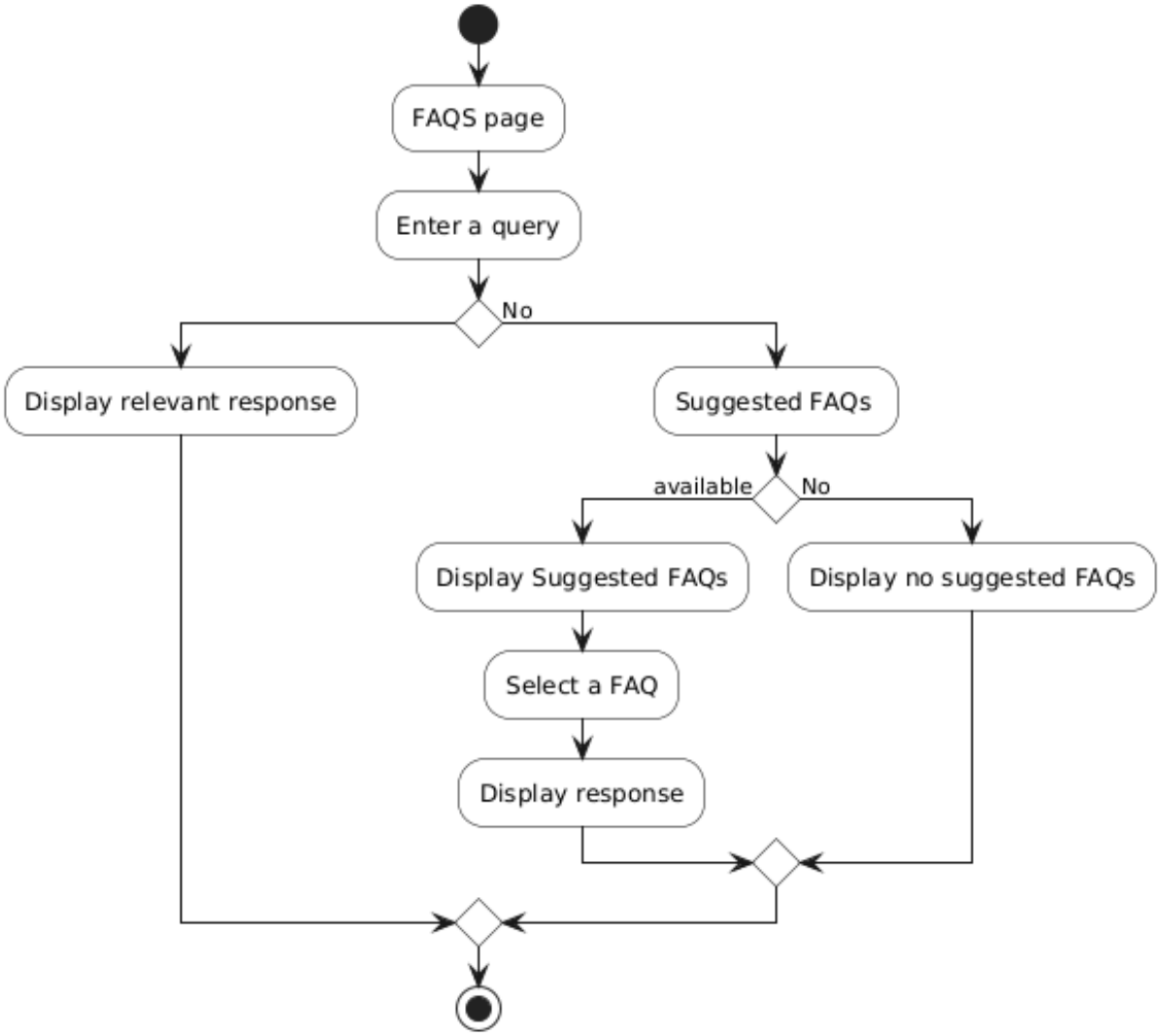


Figure 13: Activity Diagram for FAQs Chatbot

4.1.12 User-to-User Messaging and Calls

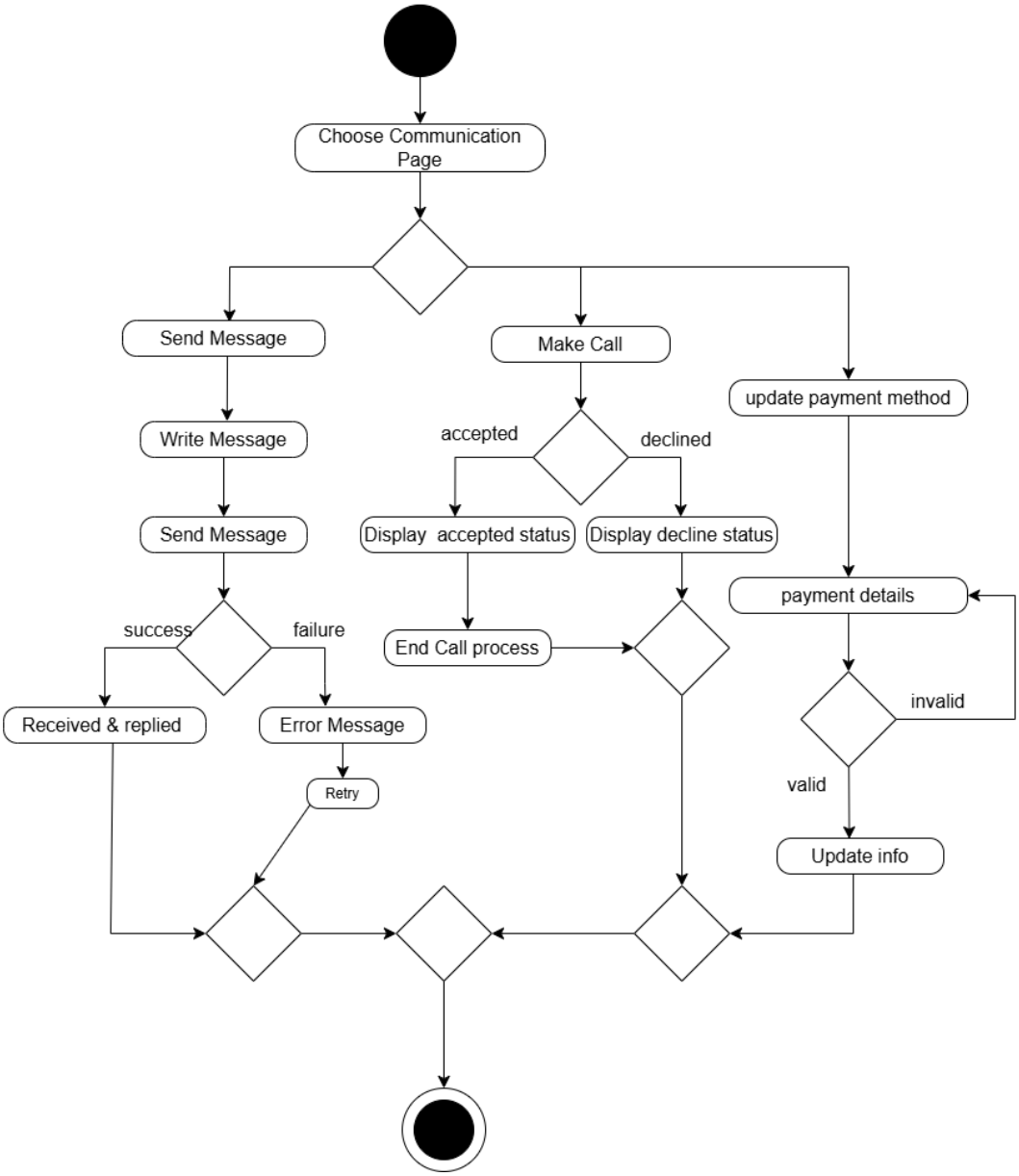


Figure 14: Activity Diagram for User-to-User Messaging and Calls

4.1.13 Manage Users and Subscriptions

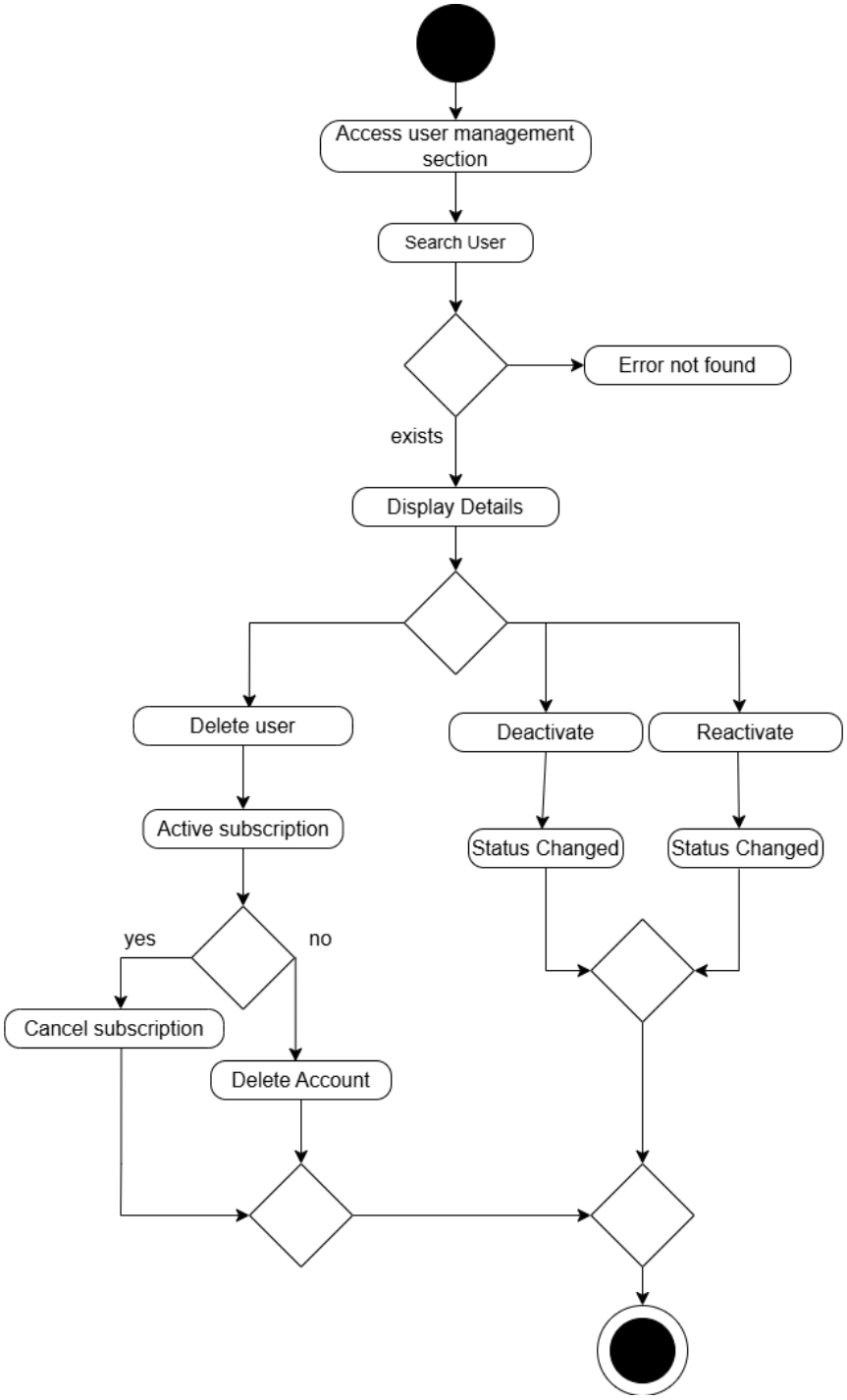


Figure 15: Activity Diagram for Managing Users and Subscriptions

4.1.14 Logout

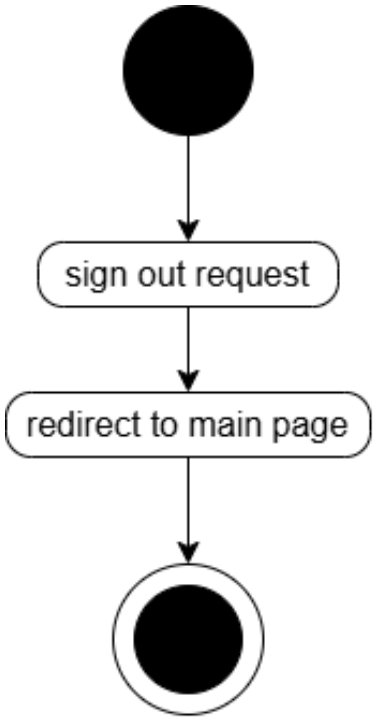


Figure 16: Activity Diagram for Sign out

4.2 Class Diagram

The following are the class entities for PodVerse:

1 User
2 Profile
3 Podcast
4 Episode
5 AIHost
6 AIGuest
7 VoiceSample
8 KnowledgeBase
9 Chunk
10 Message
11 Comment
12 Rating
13 Subscription
14 Payment
15 Notification
16 FAQsChatbot
17 FAQEntry
18 AISpeaker
19 Admin

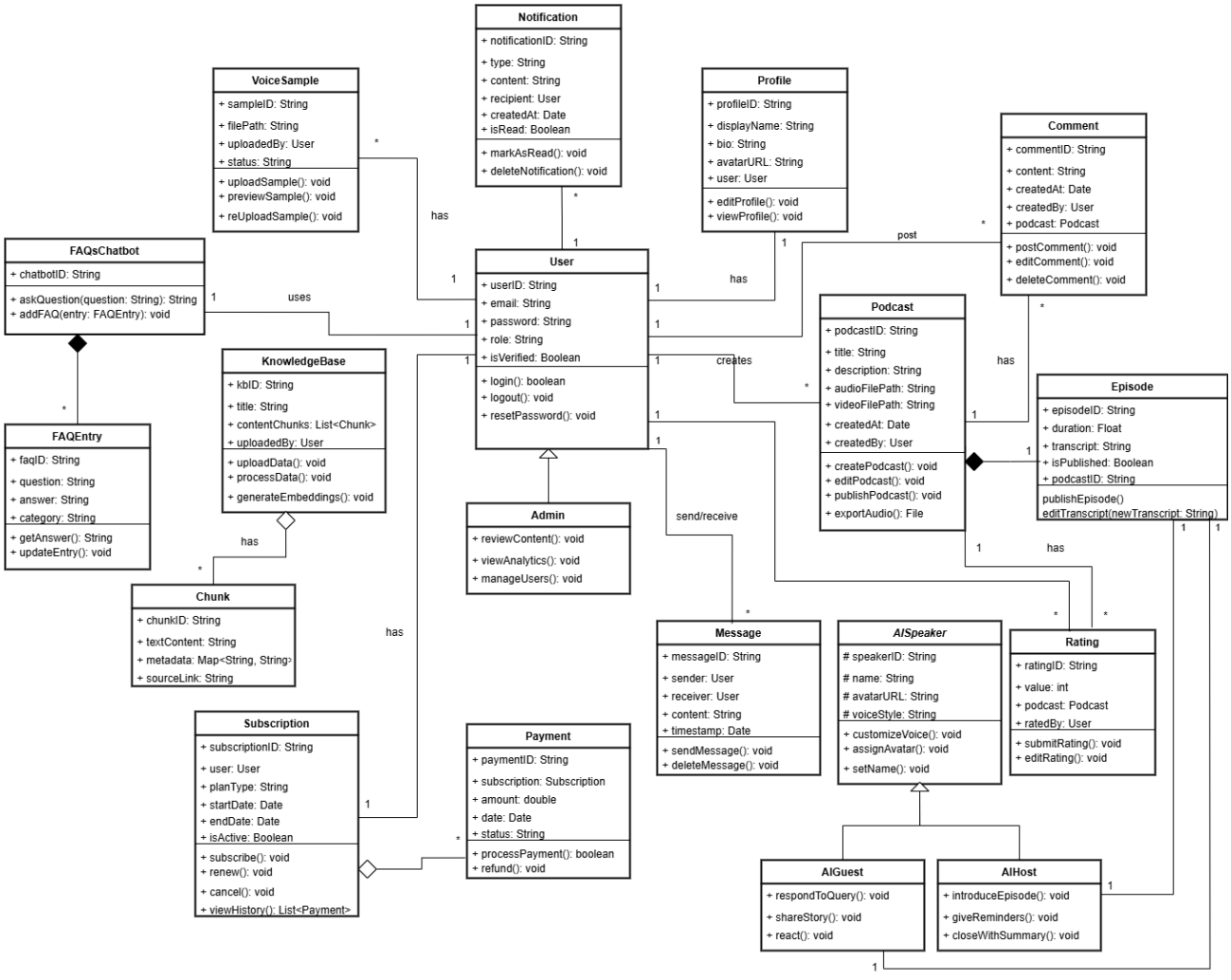


Figure 17: Class Diagram for PodVerse

4.3 Sequence Diagrams

Following are the sequence diagrams of PodVerse:

4.3.1 Module 1: User Authentication

Sequence diagram of module 1.

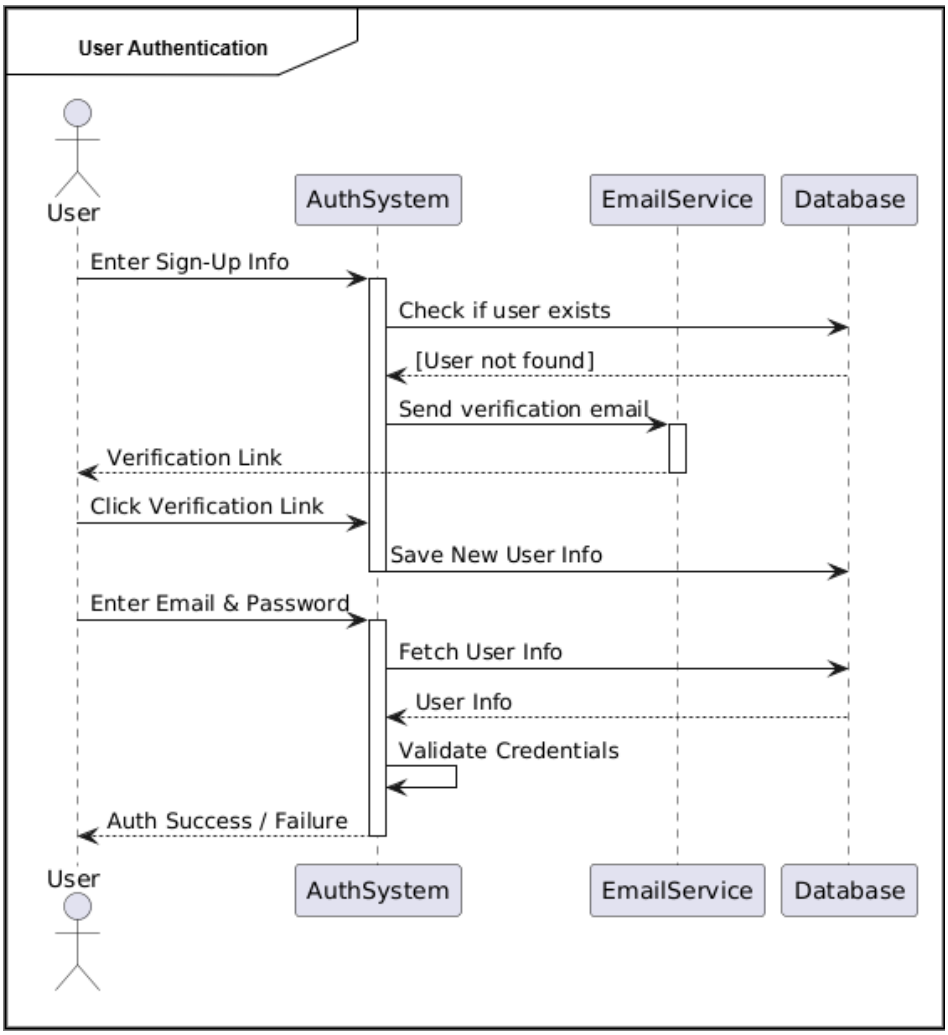


Figure 18: Sequence diagram of module 1

4.3.2 Module 2: Voice Cloning

Sequence diagram of module 2.

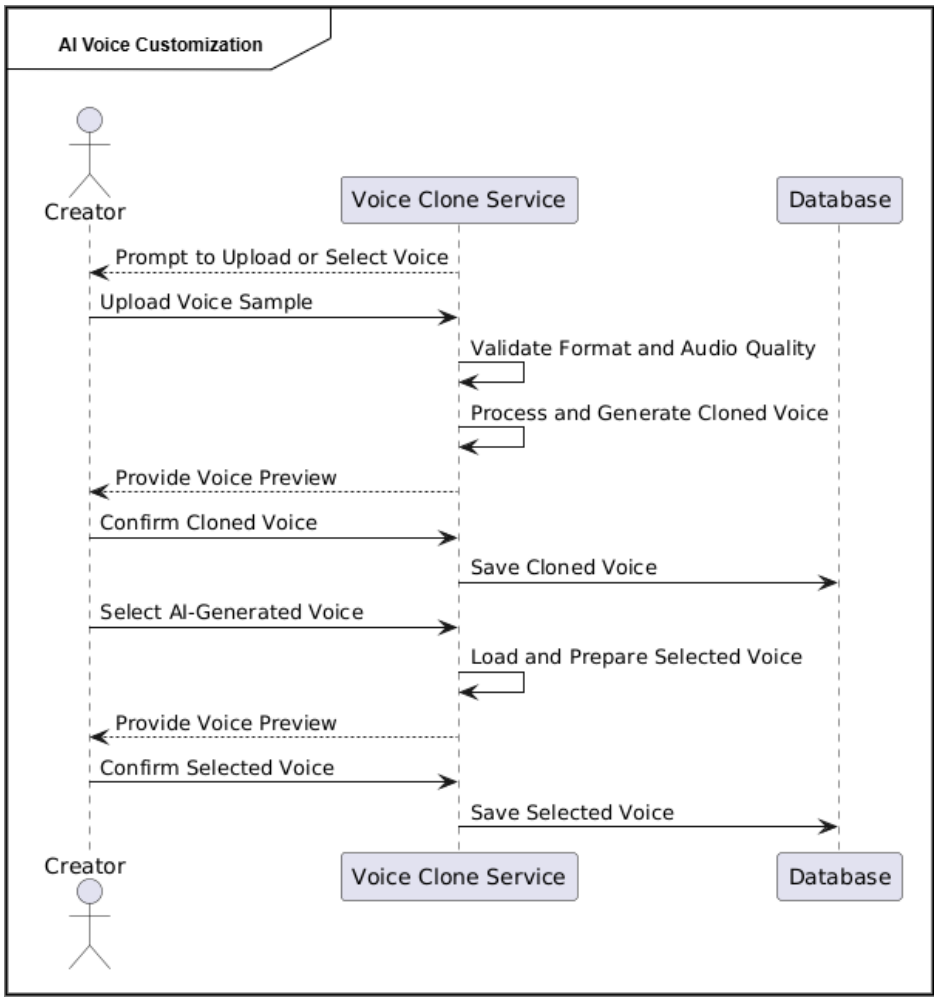


Figure 19: Sequence diagram of module 2

4.3.3 Module 3: Knowledge Base Setup

Sequence diagram of module 3.

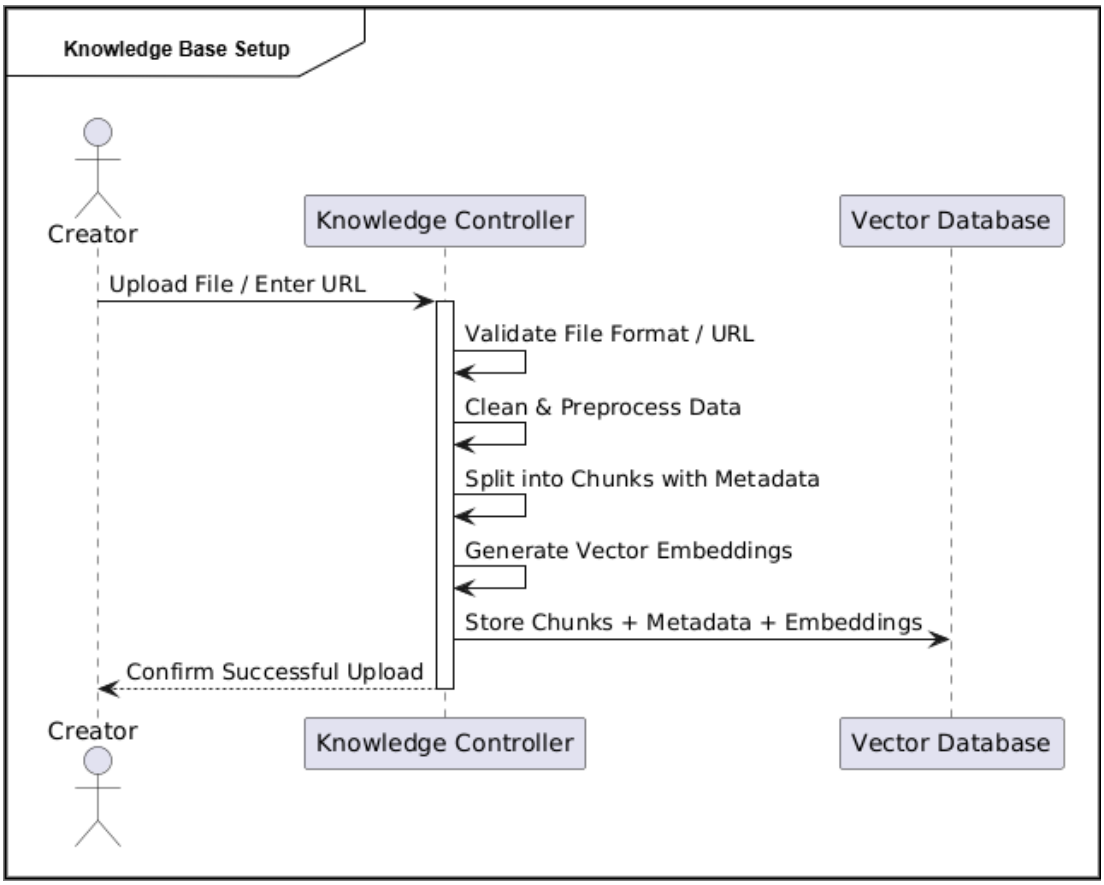


Figure 20: Sequence diagram of module 3

4.3.4 Module 4: RAG Retrieval

Sequence diagram of module 4.

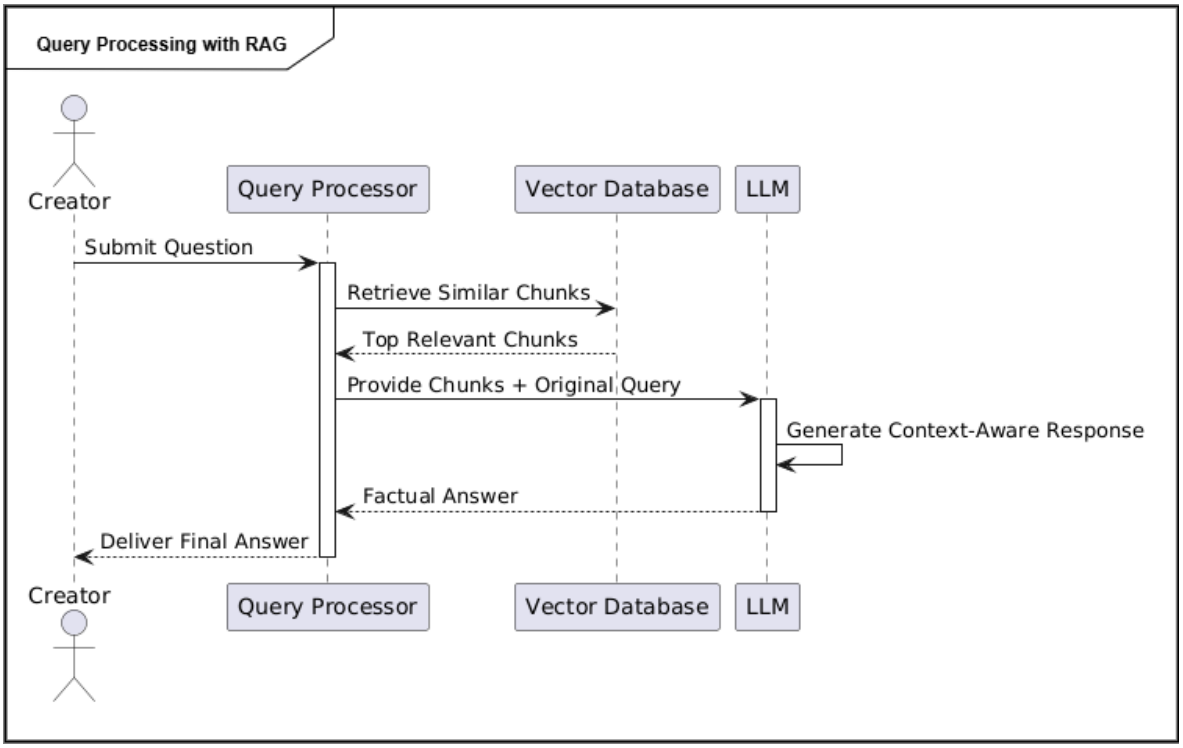


Figure 21: Sequence diagram of module 4

4.3.5 Module 6: Podcast Audio Creation

Sequence diagram of module 6.

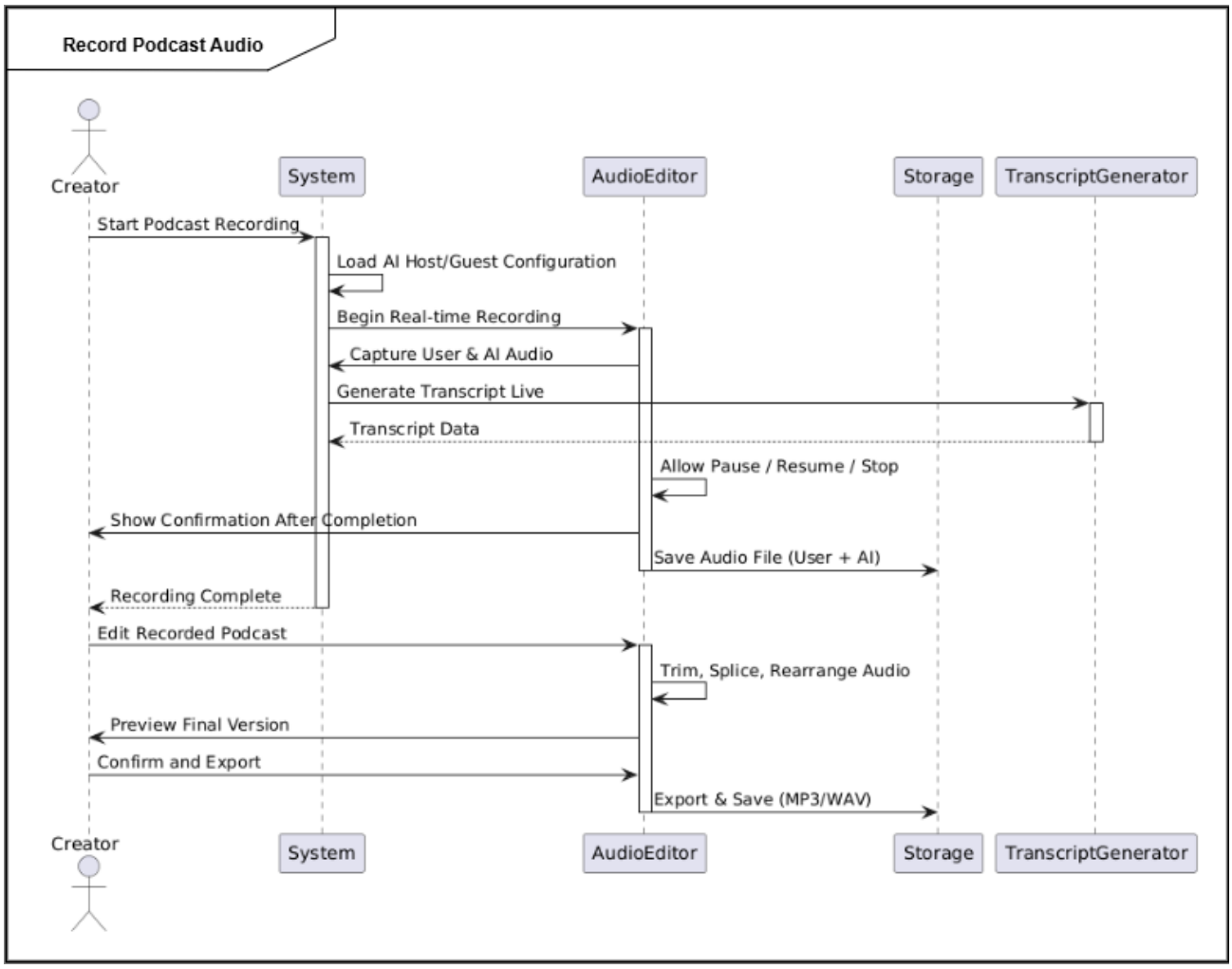


Figure 22: Sequence diagram of module 6

4.3.6 Module 9: Podcast Video Creation and Publishing

Sequence diagram of module 9.

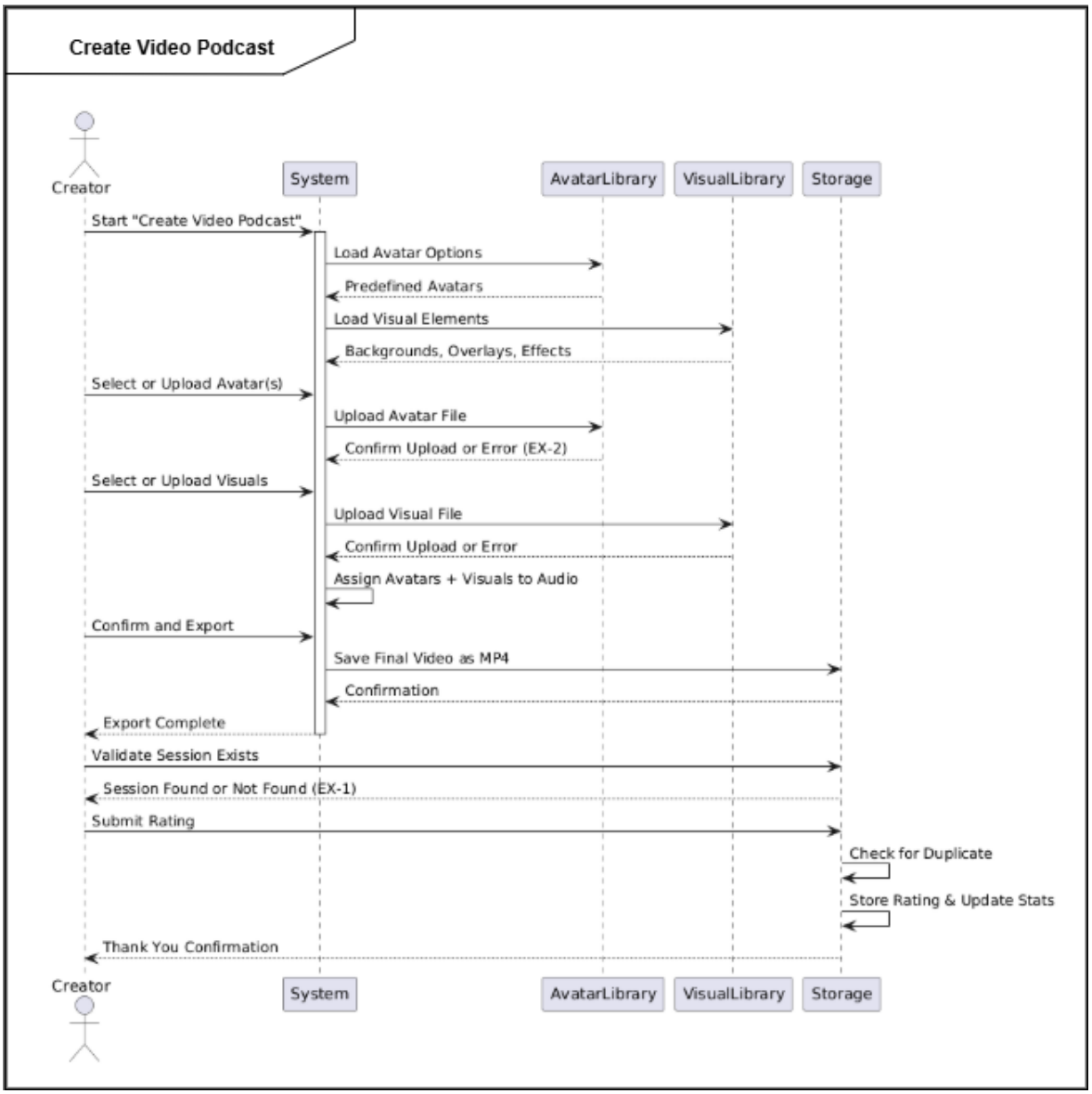


Figure 23: Sequence diagram of module 9

4.3.7 Module 10: Subscription and Payment

Sequence diagram of module 10.

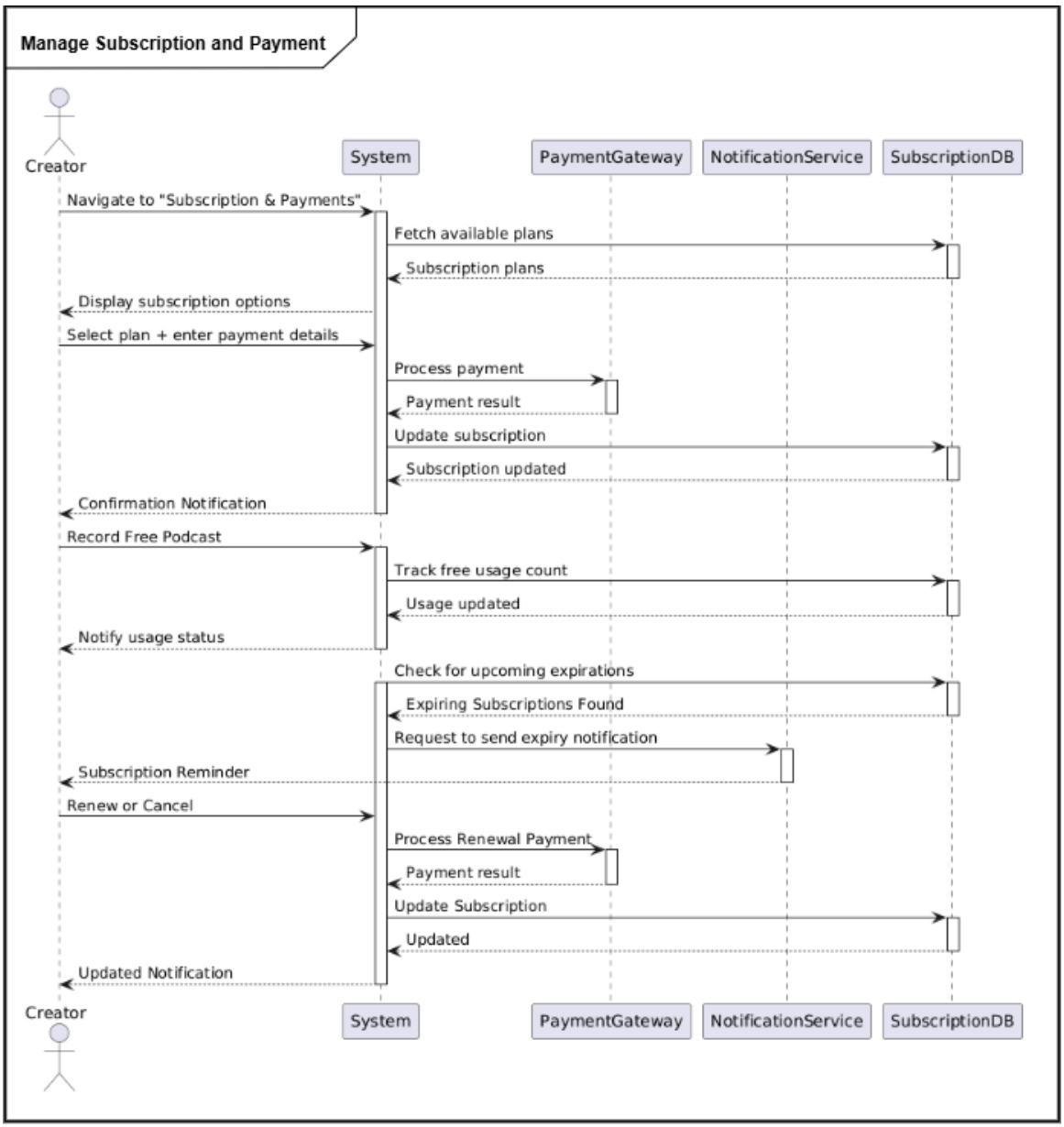


Figure 24: Sequence diagram of module 10

4.3.8 Module 11: FAQs Chatbot

Sequence diagram of module 11.

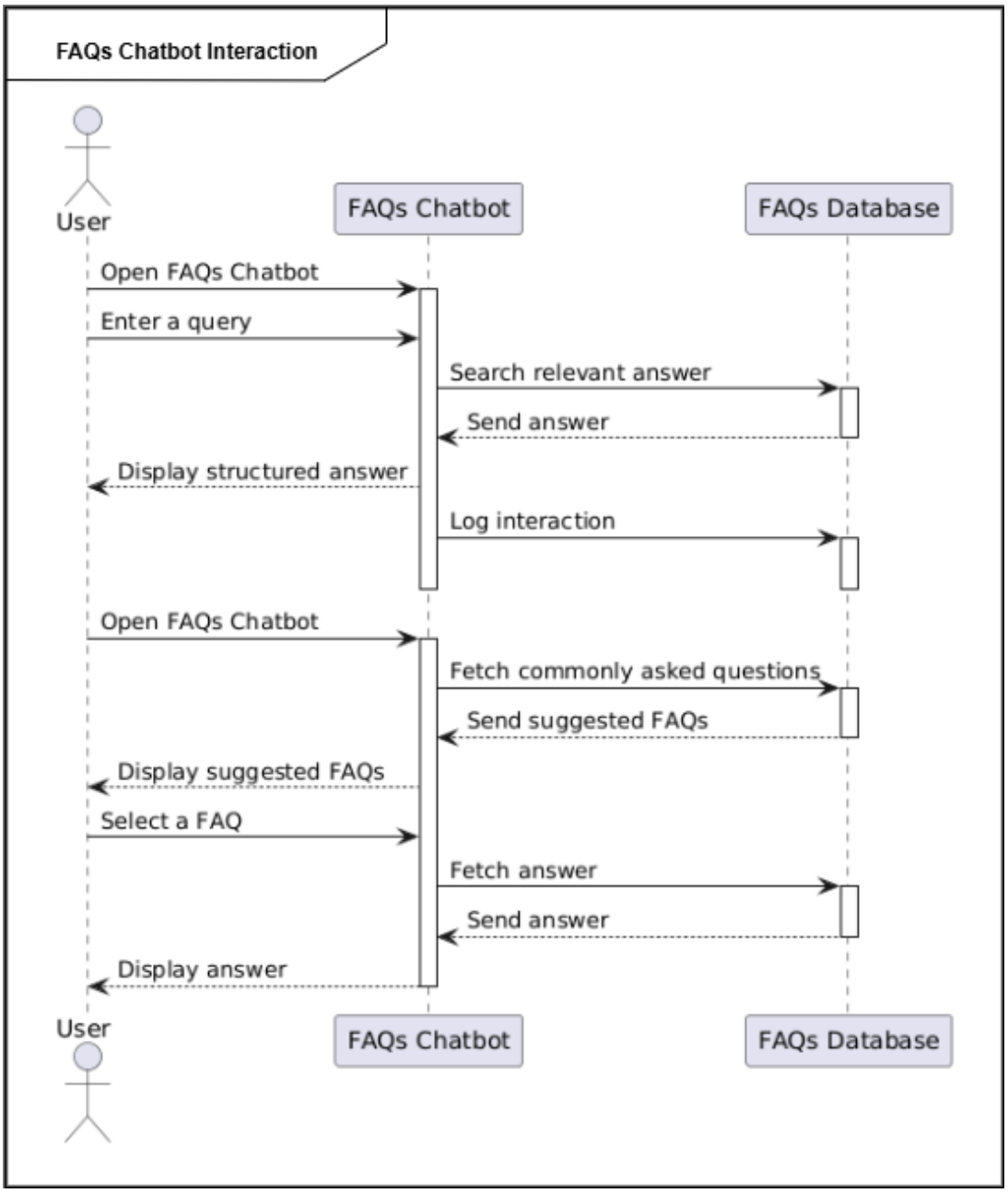


Figure 25: Sequence diagram of module 11

4.3.9 Module 12: Audience Engagement

Sequence diagram of module 12.

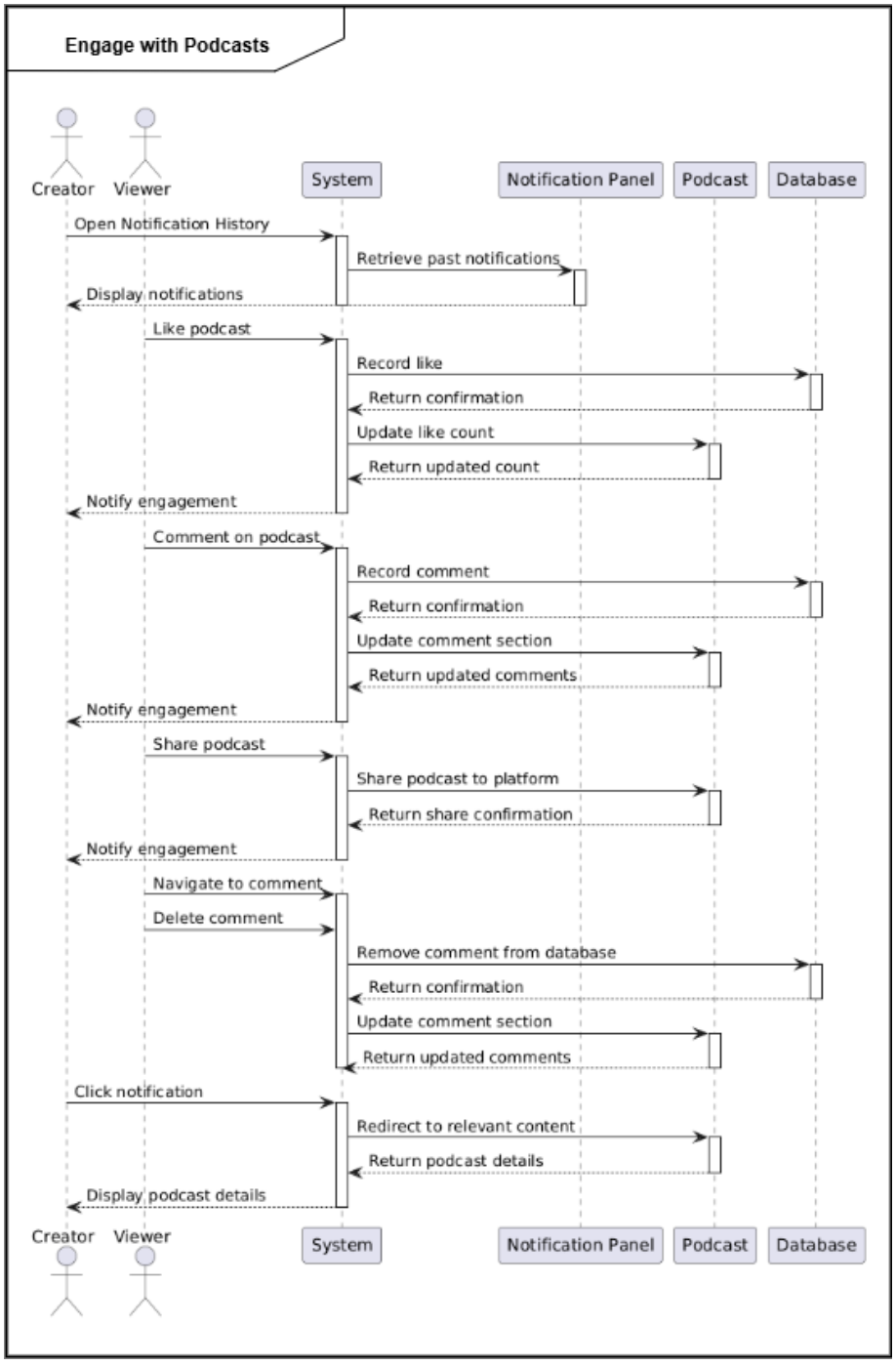


Figure 26: Sequence diagram of module 12

4.3.10 Module 13: Admin Panel and Analytics

Sequence diagram of module 13.

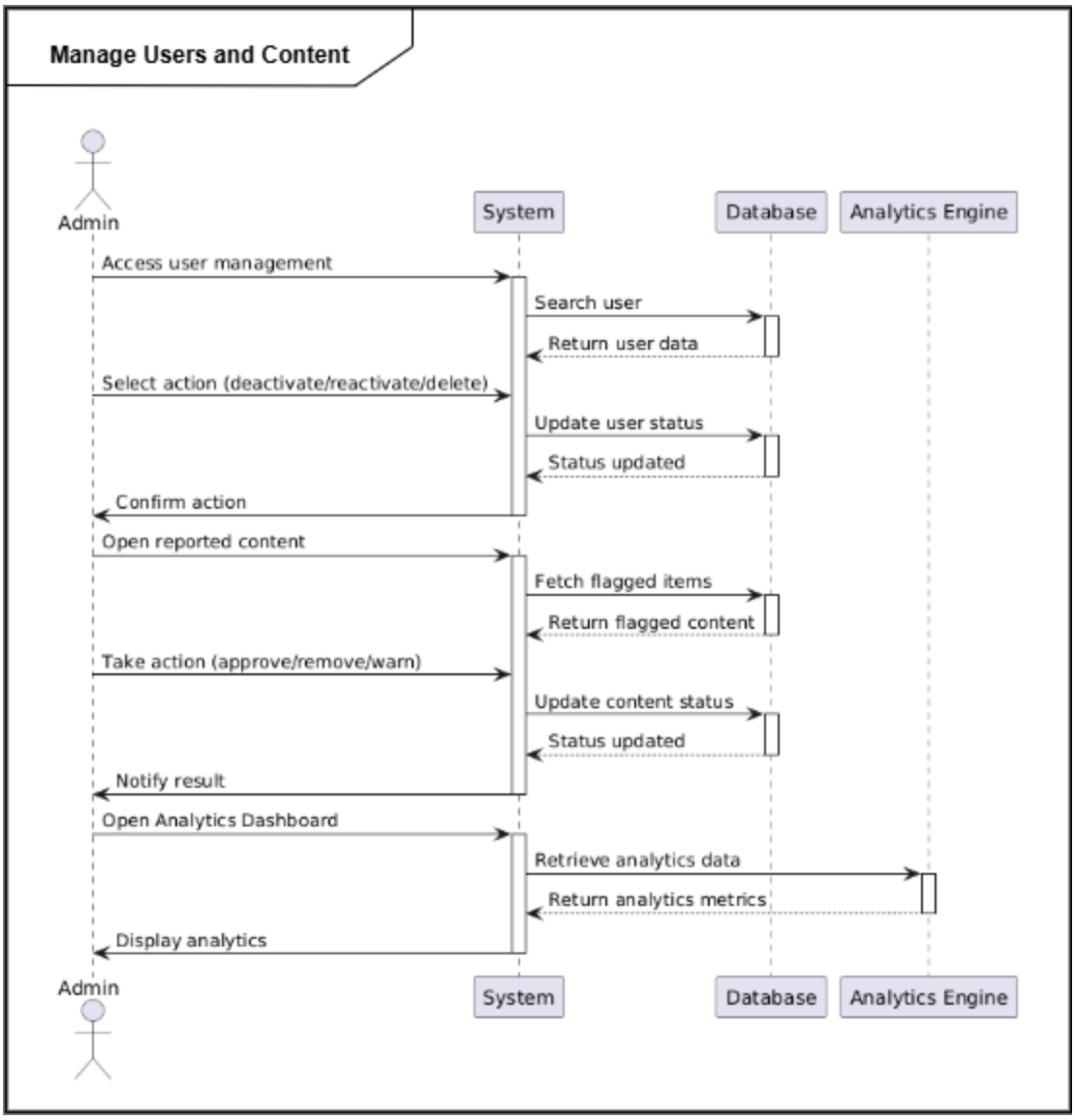


Figure 27: Sequence diagram of module 13

4.4 State Transition Diagrams

The following are the state transition diagrams of the backend processes of our system:

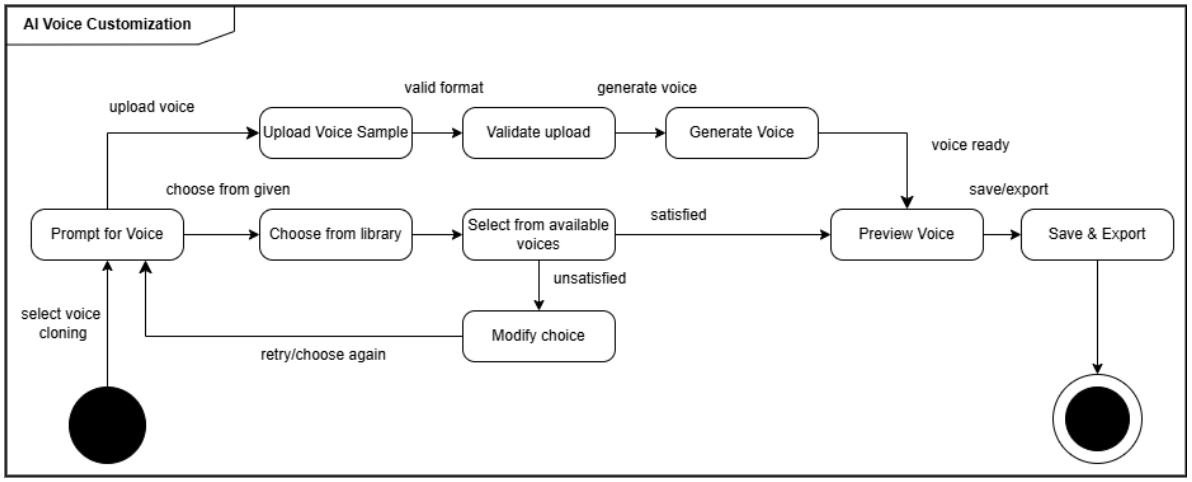


Figure 28: State transition diagram of AI Voice Customization

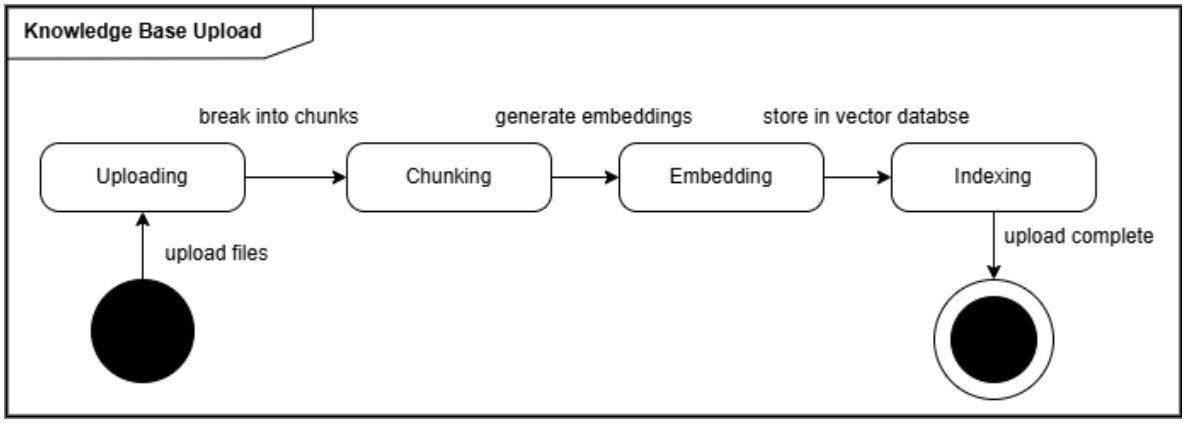


Figure 29: State transition diagram of Knowledge Base Upload

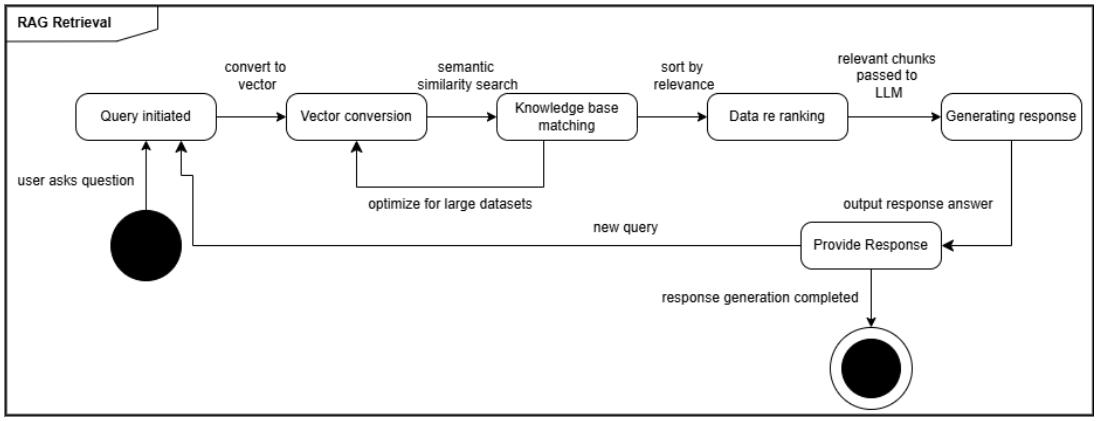


Figure 30: State transition diagram of RAG Retrieval

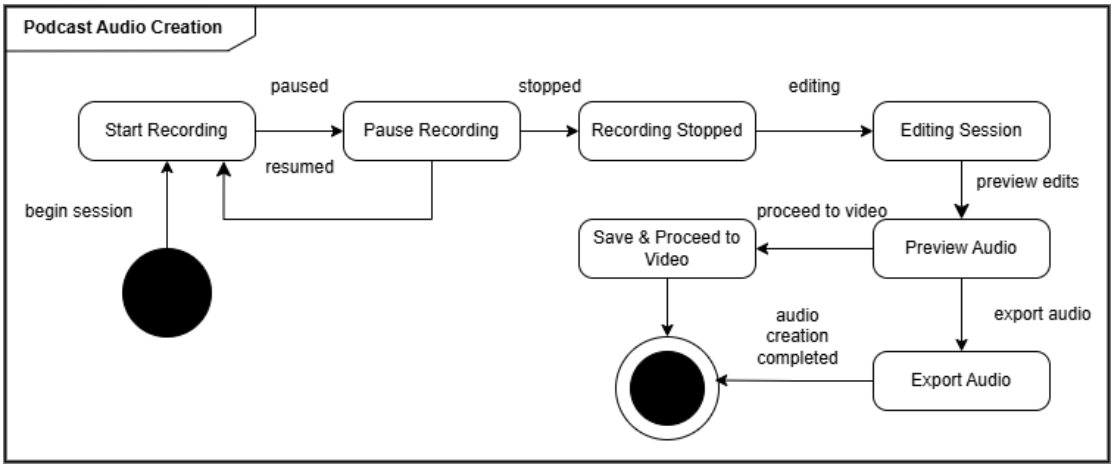


Figure 31: State transition diagram of Podcast Audio Creation

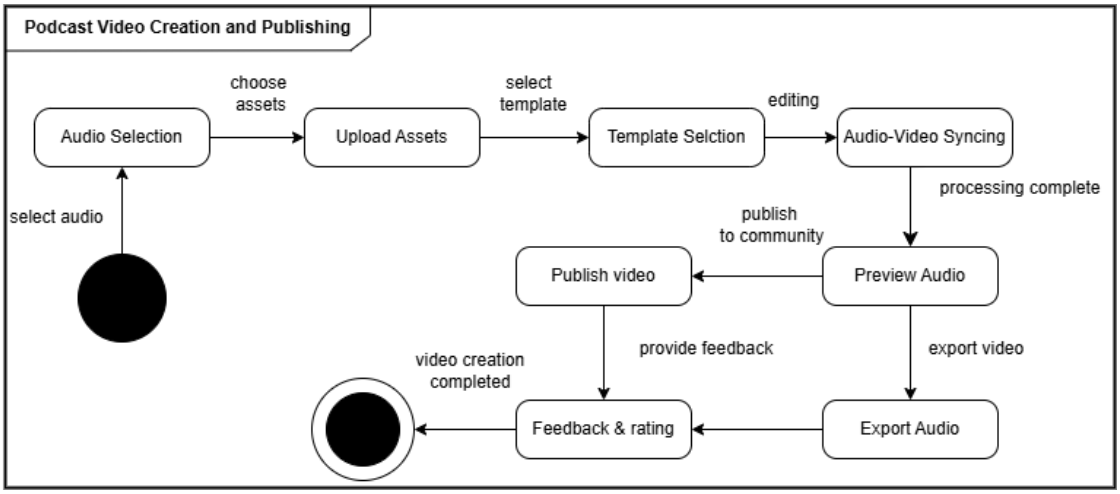


Figure 32: State transition diagram of Podcast Video Creation and Publishing

5. Data Design

The data design of PodVerse is as follows:

5.1 Data Dictionary

Following is the data dictionary of PodVerse:

5.1.1 User Collection

Table 1: Data Collection of 'User'

Field Name	Data Type	Constraints	Description
userID	String	Primary Key	Unique identifier for the user
email	String	Unique, Not Null	User's email address
fullName	String	None	User's full name
password	String	Not Null	Hashed password for secure login
profilePicUrl	String	None	URL of the user's profile image
role	String	None	Regular, Admin
createdAt	DateTime	None	Account creation date
bio	String	None	User's biography
isSubscribed	Boolean	None	Subscription status

5.1.2 Podcast Collection

Table 2: Data Collection of 'Podcast'

Field Name	Data Type	Constraints	Description
podcastID	String	Primary Key	Unique identifier for the user
creatorBy	String	Foreign Key	User ID of the podcast creator
title	String	Not Null	Title of the podcast

description	String	None	Description of the podcast
category	String	None	Category/genre of the podcast
audioFilePath	String	None	URL to the podcast audio file
videoFilePath	String	None	URL to the podcast video file
transcript	String	None	Transcript generated during recording
coverImageUrl	String	None	Podcast thumbnail image URL
createdAt	DateTime	None	Date and time the podcast was published

5.1.3 Voice Sample Collection

Table 3: Data Collection of 'Voice Sample'

Field Name	**Data Type**	**Constraints**	**Description**
sampleID	String	Primary Key	Unique identifier for sample
uploadedBy	String	Foreign Key	Linked user ID
sampleAudioUrl	String	None	URL of uploaded voice sample
cloneModelID	String	None	ID of the trained clone model
createdAt	DateTime	None	Timestamp of creation

5.1.4 Knowledge Base Collection

Table 4: Data Collection of 'Knowledge Base'

Field Name	**Data Type**	**Constraints**	**Description**
knowledgeID	String	Primary Key	Unique knowledge base ID
uploadedBy	String	Foreign Key	Linked user ID
title	String	None	Title of the knowledge item

sourceType	String	None	PDFs, URLs, CSV, etc.
content	Text	None	Original content
cleanedContent	Text	None	Pre-processed version of the content
vectorID	String	None	Associated vector representation ID
uploadedAt	DateTime	None	Upload timestamp

5.1.5 AI Speaker Collection

Table 5: Data Collection of 'AI Speaker'

Field Name	Data Type	Constraints	Description
speakerID	String	Primary Key	Unique ID of the AI Speaker
name	String	Not Null	AI speaker name
language	String	Not Null	Language spoken
voiceStyle	String	Nullable	Style of speaking (e.g., Calm, Energetic)
avatarUrl	String	Nullable	Avatar image or 3D model link
createdAt	DateTime	Not Null	Creation timestamp

5.1.6 Subscription Collection

Table 6: Data Collection of 'Subscription'

Field Name	Data Type	Constraints	Description
subscriptionID	String	Primary Key	Unique ID of the subscription
UserID	String	Foreign Key	Subscribed user's ID
planType	String	Not Null	Plan (Free, Pro, Premium)

startDate	DateTime	None	Subscription start date
endDate	DateTime	None	Subscription end/renewal date
isActive	Boolean	Default: true	Whether the subscription is currently active

5.1.7 Notification Collection

Table 7: Data Collection of 'Notification'

Field Name	**Data Type**	**Constraints**	**Description**
notificationID	String	Primary Key	Unique notification ID
userID	String	Foreign Key	Recipient user ID
title	String	None	Title of the notification
message	String	None	Main content of the notification
createdAt	DateTime	None	Timestamp of the notification
isRead	Boolean	Default: false	Read/unread status

5.1.8 Payment Collection

Table 8: Data Collection of 'Payment'

Field Name	**Data Type**	**Constraints**	**Description**
paymentID	String	Primary Key	Unique ID of the payment
userID	String	Foreign Key	User who made the payment
subscriptionID	String	Foreign Key	Related subscription
amount	Float	Not Null	Amount paid
status	String	Not Null	Payment status

paymentDate	DateTime	Not Null	Date and time of payment

5.1.9 FAQs Chatbot Collection

Table 9: Data Collection of 'FAQs Chatbot'

Field Name	Data Type	Constraints	Description
chatbotID	String	Primary Key	Unique chatbot session ID
userID	String	Foreign Key	User who started the session
sessionStart	DateTime	Not Null	When the session began
sessionEnd	DateTime	Nullable	When the session ended

5.1.10 FAQs Entry Collection

Table 10: Data Collection of 'FAQs Entry'

Field Name	Data Type	Constraints	Description
faqID	String	Primary Key	Unique ID of the FAQ entry
question	String	Not Null	FAQ question
answer	Text	Not Null	FAQ answer
category	String	Nullable	Category (e.g., Billing, Technical, General)
createdAt	DateTime	Not Null	Entry creation timestamp

5.1.11 Comment Collection

Table 11: Data Collection of 'Comment'

Field Name	Data Type	Constraints	Description
commentID	String	Primary Key	Unique identifier for the comment
userID	String	Foreign Key	User who made the comment
episodeID	String	Foreign Key	Episode the comment is associated with
content	Text	Not Null	The actual comment text
createdAt	DateTime	Not Null	Timestamp when the comment was created
updatedAt	DateTime	Nullable	Timestamp when the comment was last edited

5.1.12 Rating Collection

Table 12: Data Collection of 'Rating'

Field Name	Data Type	Constraints	Description
ratingID	String	Primary Key	Unique identifier for the rating
userID	String	Foreign Key	User who gave the rating
episodeID	String	Foreign Key	Episode that was rated
score	Integer	Not Null	Rating score (e.g., 1 to 5)
comment	String	Nullable	Optional comment/review associated with the rating
createdAt	DateTime	Not Null	When the rating was submitted

5.1.13 Message Collection

Table 13: Data Collection of 'Message'

Field Name	Data Type	Constraints	Description
messageID	String	Primary Key	Unique message ID
senderID	String	Foreign Key	User who sent the message
receiverID	String	Foreign Key	User who received the message
content	Text	Not Null	Message text
sentAt	DateTime	Not Null	Timestamp when the message was sent
isRead	Boolean	Default: false	Indicates if the message has been read

6. Implementation

This section contains the implementation details of the system. It includes algorithms, external APIs used by the system, user interface design and deployment details.

6.1 Algorithm

The following are the algorithms used in PodVerse:

Algorithm 1 Registration

Input: user_details (email, password)

Output: Registration status (success or error message)

1. Initialize database_connection
2. Input user_details (email, password, role).
3. Check if role is valid (Applicant or Recruiter):
4. If not valid, return: "Invalid role".
5. Normalize the email to lowercase.
6. Check if the email already exists using checkEmailExists(email):
7. If it exists, return "Email already exists".
8. Encrypt the password using bcrypt (encryptPassword(password)).
9. Save user_details (email and password) to the database.
10. Send a verification email using sendVerificationCode(email).
11. Return "Registration successful. Please verify your email.".

Algorithm 1.1 checkUsernameExists	**Algorithm 1.2: encryptPassword**
Input: username	**Input: password**

Output: boolean value	Output: encrypted_password
1: initialize database_connection 2: input email 3: user_record ← query database for user where email = input email 4: if (user_record exists) then 5: return true 6: else 7: return false	1: initialize encryption_algorithm 2: input password 3: encrypted_password ← encryption_algorithm.encrypt(password) 4: return encrypted_password

Algorithm 2 Login

Input: email,password

Output: Login status (success or error message)

1. Initialize database_connection.
2. Input email and password.
3. Normalize the email to lowercase.
4. Query the database for a user with the given email.
5. If no user is found, return "Invalid email".
6. Compare the provided password with the stored password using bcrypt.compare.
7. If the password is incorrect, return "Invalid password".
8. Check if the user is verified (isVerified).
9. If not verified, return "Email not verified".
10. Generate a secure token (setUser(user)).
11. Store the token as a cookie in the response.
12. Return "Login successful".

Algorithm 3 Voice Cloning	**Algorithm 4 TranscribePodcastAudio**
Input: User's audio sample (.wav)	**Input: audio_file (.wav)**
Output: Cloned voice model ID	**Output: Transcript text**
1: Import necessary libraries (torchaudio, numpy, os, torch) 2: Function preprocess_audio(audio_sample): 3: Convert to mono channel, 16kHz if not already 4: Return cleaned_audio 5: Function extract_embedding(cleaned_audio): 6: Load Tortoise TTS encoder model 7: embedding ← encoder(cleaned_audio) 8: Return embedding 9: Function save_embedding(user_id, embedding): 10: path ← "voice_embeddings/" + user_id + ".pt" 11: Save embedding tensor to path using torch.save() 12: Return path 13: Main Function clone_voice(user_id, audio_sample): 14: cleaned_audio ← preprocess_audio(audio_sample) 15: embedding ← extract_embedding(cleaned_audio) 16: embedding_path ← save_embedding(user_id, embedding) 17: Save embedding_path in MongoDB user profile 18: Return "Voice cloned successfully", embedding_path	1: Import required libraries (torchaudio, os, whisper or other STT model) 2: Function load_stt_model(): 3: model ← load Whisper / Wav2Vec2 model locally 4: Return model 5: Function preprocess_audio(audio_file): 6: Convert audio to 16kHz mono WAV format if needed 7: Return cleaned_audio_path 8: Function transcribe_audio(model, cleaned_audio): 9: transcript ← model.transcribe(cleaned_audio) 10: Return transcript 11: Main Function transcribe_podcast(audio_file): 12: model ← load_stt_model() 13: cleaned_audio ← preprocess_audio(audio_file) 14: transcript ← transcribe_audio(model, cleaned_audio) 15: Save transcript to MongoDB with reference to podcast_id 16: Return transcript
Algorithm 5 GenerateTranscriptSummary	**Algorithm 6 BasicAIUserConversation**
Input: transcript_text	**Input: speaker_turns, conversation_text, user_id**
Output: summary_text	**Output: Generated podcast audio file path**
1: Import HuggingFace transformers library 2: Function load_summarizer_model(): 3: model_name ← "facebook/bart-large-cnn" OR "google/flan-t5-base" 4: tokenizer ← load tokenizer for model_name 5: model ← load pre-trained model from HuggingFace 6: Return tokenizer, model 7: Function preprocess_transcript(transcript_text): 8: Clean special characters and limit to max 1024 tokens 9: Return cleaned_text 10: Function generate_summary(tokenizer, model, input_text): 11: inputs ← tokenizer(input_text, return_tensors="pt", truncation=True)	1: Import required libraries (TTS model, pyaudio, ffmpeg, torchaudio) 2: Function synthesize_speaker_audio(text, speaker_embedding): 3: Generate waveform using Tortoise or Tacotron with speaker_embedding 4: Return audio_waveform 5: Function merge_conversation_turns(conversation_list): 6: For each (speaker_role, sentence) in conversation_list: 7: speaker_embedding ← load_embedding(speaker_role) 8: audio ← synthesize_speaker_audio(sentence, speaker_embedding) 9: Append audio to full_session_audio 10: Return full_session_audio 11: Main Function record_ai_podcast(conversation_list, user_id):

```
12:  summary_ids ← model.generate(inputs["input_ids"], max_length=150, min_length=40)
13:  summary ← tokenizer.decode(summary_ids[0], skip_special_tokens=True)
14:  Return summary

15: Main Function summarize_transcript(transcript_text):
16:  tokenizer, model ← load_summarizer_model()
17:  input_text ← preprocess_transcript(transcript_text)
18:  summary ← generate_summary(tokenizer, model, input_text)
19:  Save summary in MongoDB linked to transcript_id
20:  Return summary
```

```
12:  full_audio ← merge_conversation_turns(conversation_list)
13:  file_name ← "user_id_timestamp_ai_session.wav"
14:  Save full_audio to Firebase Storage under `podcasts/user_id/`
15:  Create podcast metadata (title, timestamp, duration, roles)
16:  Save metadata in MongoDB `podcasts` collection
17:  Return "Podcast recorded successfully", file path
```

6.2 External APIs/SDKs

Below are the APIs used in PodVerse :

Table 14: Details of APIs used in the project

Name of API and version	**Description of API**	**Purpose of usage**	**List down the API endpoint/function/class in which it is used**
Stripe (version 2020-08-27)	Credit Card payment integration	Sandbox used for the orders payment	stripe.paymentMethods.create
Cloudinary	Image and Video management solution	Uploading Product Images on Cloudinary server	https://api.cloudinary.com/v1_1/demo/image/upload

6.3 User Interface

Details about user interface with descriptions. Provide the User Interface for each sub-system (such as Mobile App, Web App, Client App, Admin App). Provide description of each User Interface explaining the details.
When inserting User Interfaces, use appropriate size of the image, for example, for mobile app, 2-4 screens can be placed on a single page.

Following are few examples of User Interfaces:

7.3.1 Login Screen

Login screen of our mobile app where user have to choose its role and its company.

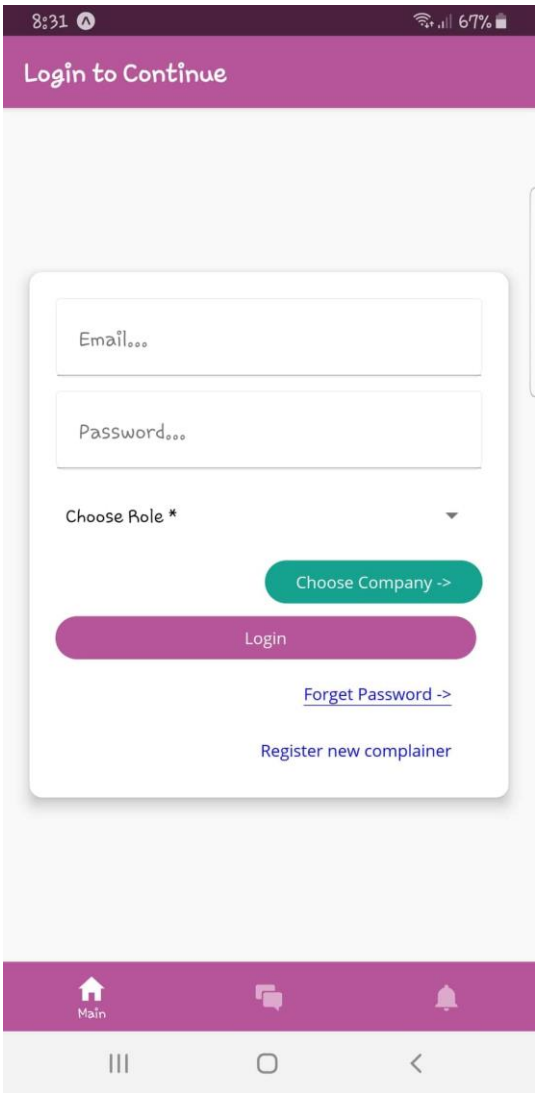


Figure 2 Login Screen

Home Screen

Home screen where total, delayed and Other complaints are shown.

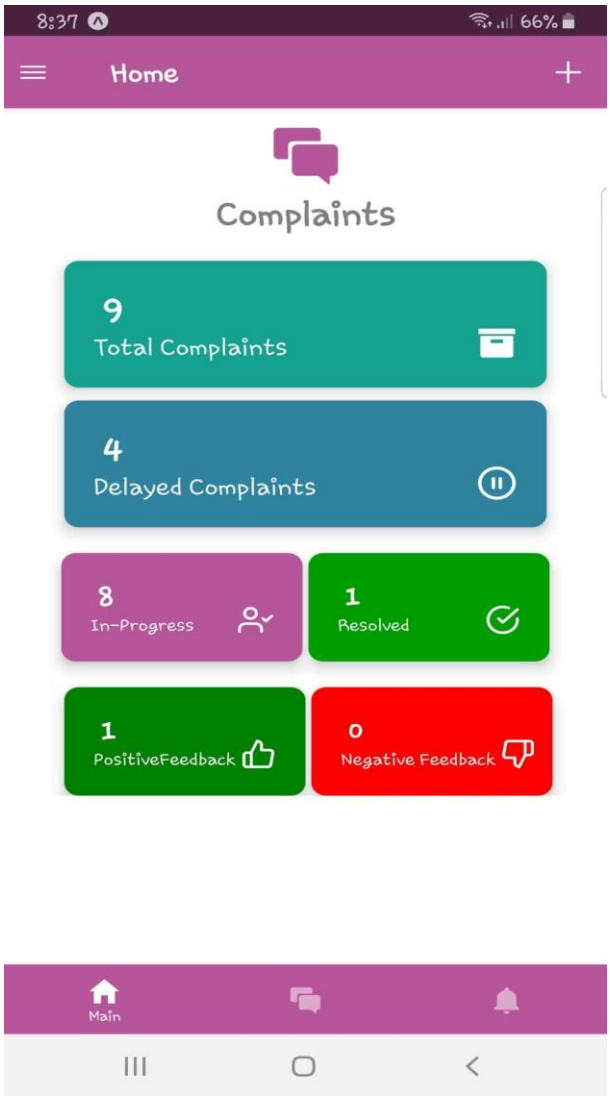


Figure 1Home Screen

7.3.2 Assignee Dashboard

Complain Assignee can view the graphs of month-wise complains, Resolved complains, summary and a list of submitted complains.

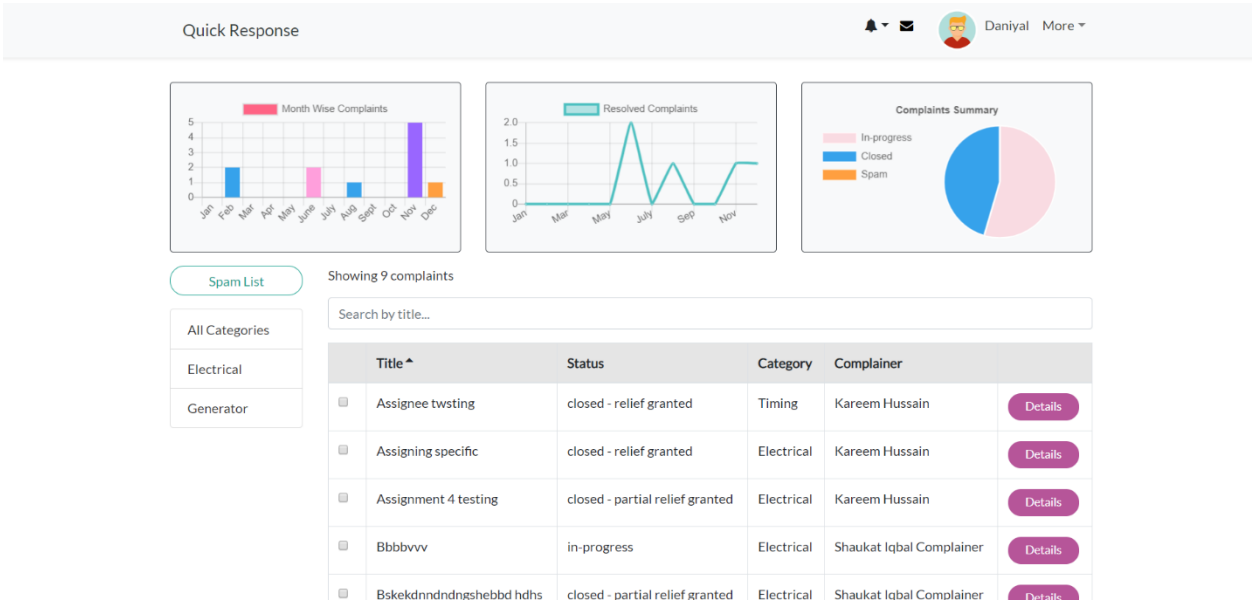


Figure 3 Assignee Dashboard

7.3.3 New Complaint

Complain Assignee and Complainer can create a new complain by providing Description, Category, Title, Location etc.

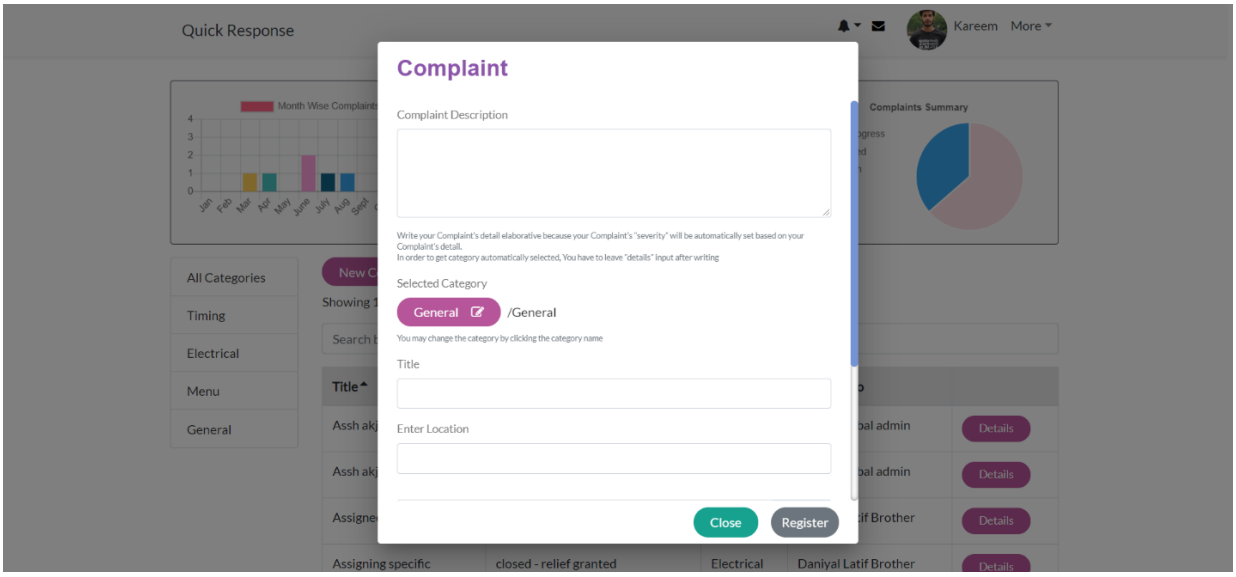


Figure 4 New Complaint

6.4 Deployment

Specify the deployment environments used for hosting and live testing of all the sub-systems of the project. Provide the details of hosting/cloud service used, set of available software and their versions used etc.

7. Testing and Evaluation

Once the system has been successfully developed, testing has to be performed to ensure that the system working as intended. This is also to check that the system meets the requirements stated earlier. Besides that, system testing will help in finding the errors that may be hidden from the user. The testing must be completed before it is deployed for use.

There are few types of testing which includes the unit testing, functional testing and integration testing.
You are *required* to perform each of these in-depth to ensure system quality.

7.1 Unit Testing

It's a level of software testing where individual units of a software/component are tested. The purpose is to validate that each unit of the software performs as designed.

Unit Testing 1: Login as Patient with valid and invalid credentials
Testing Objective: To ensure the login form is working correctly with valid and invalid credentials/inputs.

No.	Test case/Test script	Attribute and value	Expected result	Result
1	Check the email field of login to validate that it takes proper email	Email: abc@gmail.com	Validates email address and moves cursor to next textbox	Pass
2	Check the email field of login to validate that it displays error message.	Email: abc.gmail.com	Highlights field and displays error message	Pass

7.2 Functional Testing

The functional testing will take place after the unit testing. In this functional testing, the functionality of each of the module is tested. This is to ensure that the system produced meets the specifications and requirements.

Functional Testing 1: Login with different roles (Management, Patient, Doctor)
Objective: To ensure that the correct page with the correct navigation bar is loaded.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1.	Login as a 'Management' member.	Username: (correct username M003) Password:	Main page for the Management is loaded with the	Logged in and redirected to	Pass

		(correct password 1234)	Management navigation bar.	management main page.	
2.	Login as a 'Doctor' member.	Username: D003 Password: 1234	Main page for the Doctor is loaded with the doctor navigation bar.	Login failed – invalid credentials error	Fail

7.3 Business Rules Testing

Decision table based testing technique is used to test business rules. The business rules were defined in FRs and Use Cases
Decision based testing uses a systematic approach where input and outputs are provided in tabular form. It is a precise and compact way to model complicated logic. The table contains conditions and actions are used for test cases where conditions as inputs and actions as outputs.

Detailed example is as given in Appendix E.

7.4 Integration Testing

Integration tests assess whether a set of classes that must work together do so without error. They ensure that the interfaces and linkages between different parts of the system work properly. At this point, the classes have passed their individual unit tests, so the focus now is on the flow of control among the classes and on the data exchanged among them. Integration testing follows the same general procedures as unit testing: The tester develops a test plan that has a series of tests, which, in turn, have a test. Integration testing is often done by a set of programmers and/or systems analysts.

Integration Testing 1: Scheduling Patient Appointment
Testing Objective: To ensure the scheduling is being done correctly and *the* ***interface*** *between* module 'Patient/Doctor Management' and module 'Appointment/Scheduling' *is running correctly*.

No.	Test case/Test script	Attribute and value	Expected result	Actual result	Result
1.	Make Appointment	Doctor schedule, patient preferred date and time	Successfully create doctor-patient appointment showing date and time of appointment.	Appointment created successfully	Pass
2.	Change Appointment booking	Select Date and time	Final date and time of appointment will be shown.	Appointment time changed successfully	Pass

Appendix D: General Coding Standards & Guidelines

1. Follow a consistent variable naming convention throughout the code. E.g.
    - Snakecase (words are delimited by "_" like: variable_one)
    - Pascalcase (words are delimited by capital letters like: VariableOne)
    - Camelcase (words are delimited by capital letters except the initial word like: variableOne)
    - Hungarian Notation (describes the variable type or purpose at the start of the variable name like: arrDistributeGroup)
2. Use naming that visually describes scope like privateField, Const etc
3. Use read only/immutable when a field's value should not be changed after initialization
4. Use only get, for properties that should not be updated from outside
5. Name functions according to their functionalities.
6. Insert appropriate comments to make the code understandable to any reader. Additionally follow a consistent style to do so. E.g.

```
/* the below function will be used for the addition of two variables*/
int Add(){
//logic of the function
}
```

   Avoid commenting on obvious things
7. Make use of indentation for indicating the start and end of the control structures along with a clear specification of where the code is between them.
8. Follow consistent naming convention for files and folders.
9. Follow proper structure for classes
10. Group code entities logically into projects/packages/modules/folders
    a. Separate logical layers of application into different modules/services/utilities etc.
    b. User separate files for each class, struct, interface, enum etc. Name of the file and the enclosing entity must be same. E.g., class Employee in Employee.cs/Employee.java
11. Define and use everything within the minimum scope possible
12. Use proper access modifier for all code entities if required
13. Code entities should have maximum cohesion and least coupling possible.
14. Follow DRY law.
    a. Do not repeat code.
    b. A piece of knowledge should exist only in one place within the codebase/application
    c. Reuse code as much as possible
    d. Always write short methods
    e. Single method should not have too many logic, long conditional flow or too many parameters
15. Strictly follow Single Responsibility Principle (SRP) when writing methods, classes, modules, projects, packages, or any other code entities.
16. Write classes and other code entities that are easy to extend without modification.
17. Handle exceptions
18. Log exception and other significant event details
19. Follow a consistent convention for logging all over the application

Appendix E: Business rules testing

Methodology for creating decision table:

1. Identify Conditions & Values	Find the data attribute each condition tests and all of the attribute's values.
2. Identify Possible Actions	Determine each independent action to be taken for the decision or policy.
3. Compute Max Number of Rules	Multiply the number of values for each condition data attribute by each other.
4. Enter All Possible Rules	Fill in the values of the condition data attributes in each numbered rule column.
5. Define Actions for each Rule	For each rule, mark the appropriate actions with an X in the decision table.
6. Verify the Policy	Review completed decision table with end-users.
7. Simplify the Table	Eliminate and/or consolidate rules to reduce the number of columns.

Example:

The provided example is of a super store.

SUPERSTORES MARKETING POLICY #1:

- Repeat customers get free shipping.
- New customers pay regular shipping on their first order.

Decision table:

Condition	Rule 1	Rule 2
Repeat customer?	Y	N
Free shipping	Y	N

SUPERSTORES MARKETING POLICY #2:

- Repeat customers get free shipping.
- New customers pay regular shipping on their first order.
- Any customer who uses the SuperStore (SS) charge card gets a 5% discount.

Decision table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Repeat customer?	Y	Y	N	N
Used SS card?	Y	N	Y	N
Free shipping				
5% discount				

SUPERSTORES MARKETING POLICY #2:

- Repeat customers get free shipping.
- New customers pay regular shipping on their first order.
- Any customer who uses the SuperStore (SS) charge card gets a 5% discount.

Decision table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4
Repeat customer?	Y	Y	N	N
Used SS card?	Y	N	Y	N
Free shipping	Y	Y	N	N
5% discount	Y	N	Y	N

SUPERSTORES MARKETING POLICY #3:

- Repeat customers get free shipping.
- New customers pay regular shipping on their first order. However, if a new customer's first order is over $100, shipping is free.
- Any customer who uses the SuperStore (SS) charge card gets a 5% discount.

Decision table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
Repeat customer?	Y	Y	Y	Y	N	N	N	N
Used SS card?	Y	Y	N	N	Y	Y	N	N
Order over $100?	Y	N	Y	N	Y	N	Y	N
Free shipping								
5% discount								

SUPERSTORES MARKETING POLICY #3:

- Repeat customers get free shipping.
- New customers pay regular shipping on their first order. However, if a new customer's first order is over $100, shipping is free.
- Any customer who uses the SuperStore (SS) charge card gets a 5% discount.

Decision table:

Condition	Rule 1	Rule 2	Rule 3	Rule 4	Rule 5	Rule 6	Rule 7	Rule 8
Repeat customer?	Y	Y	Y	Y	N	N	N	N
Used SS card?	Y	Y	N	N	Y	Y	N	N
Order over $100?	Y	N	Y	N	Y	N	Y	N
Free shipping	Y	Y	Y	Y	Y	N	Y	N
5% discount	Y	Y	N	N	Y	Y	N	N

- Now Combine rules where it is apparent that an alternative does not make a difference in the outcome



This is the final table and now you have to create test cases on every rule. In above example there are 6 rules so there shall be 6 test cases.